

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ ĐÔNG Á
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TẬP LỚN

HỌC PHẦN: TRÍ TUỆ NHÂN TẠO

MÃ ĐỀ THI: 24

**TÊN ĐỀ TÀI: XÂY DỰNG CHƯƠNG TRÌNH NHẬN DẠNG
CHỮ VIẾT DÙNG MẠNG NƠ RON**

LỚP TÍN CHỈ: TTNT.03.K13.05.LH.C04.1_LT

Giảng viên hướng dẫn: Nguyễn Phồn Lữ

Danh sách sinh viên thực hiện: Nhóm 7

TT	Mã sinh viên	Sinh viên thực hiện	Lớp hành chính
1	20223095	Nguyễn Quang Hoàng	DCCNTT13.10.16
2	20223218	Phan Đăng Huy	DCCNTT13.10.16
3	20223223	Ngô Văn Quân	DCCNTT13.10.16
4	20223051	Đào Văn Thái	DCCNTT13.10.16
5	20223160	Hoàng Thanh Tùng	DCCNTT13.10.16

MỤC LỤC

DANH MỤC HÌNH ẢNH.....	III
DANH MỤC BẢNG BIỂU.....	V
LỜI MỞ ĐẦU.....	1
CHƯƠNG I. CƠ SỞ LÝ THUYẾT.....	3
1.1. Tổng quan về trí tuệ nhân tạo.....	3
1.1.1. Lịch sử phát triển của trí tuệ nhân tạo.....	3
1.1.2. Các hướng nghiên cứu chính trong trí tuệ nhân tạo.....	3
1.1.3. Các ứng dụng tiêu biểu của trí tuệ nhân tạo.....	4
1.2. Mạng nơ ron tích chập (CNN).....	4
1.2.1. Kiến trúc cơ bản của CNN.....	4
1.2.2. Quá trình huấn luyện CNN.....	6
1.2.3. Ưu điểm của CNN.....	7
1.2.4. Nhược điểm của CNN.....	7
1.2.5. Ứng dụng của CNN.....	8
1.3. Các công cụ, thư viện được sử dụng trong đề tài.....	9
1.3.1. TensorFlow và Keras.....	9
1.3.2. TensorFlow Datasets.....	9
1.3.3. NumPy.....	9
1.3.4. OpenCV.....	9
1.3.5. Pillow.....	10
1.3.6. Tkinter.....	10
1.3.7. Matplotlib và Seaborn.....	10
CHƯƠNG II. XÂY DỰNG CHƯƠNG TRÌNH.....	11
2.1. Mô tả bài toán.....	11
2.1.1. Phân tích yêu cầu hệ thống.....	11
2.1.2. Thiết kế kiến trúc tổng quan.....	12
2.2. Mô tả chi tiết giải thuật.....	13
2.2.1. Pipeline tiền xử lý ảnh đầu vào.....	13
2.2.2. Kiến trúc mô hình CNN và huấn luyện mô hình.....	15

2.3. Cấu trúc mã nguồn.....	18
2.4. Thiết kế giao diện chương trình.....	24
2.4.1. <i>Nền tảng và triết lý thiết kế</i>	24
2.4.2. <i>Bố cục và thành phần giao diện</i>	25
2.4.3. <i>Luồng tương tác và trải nghiệm người dùng</i>	26
CHƯƠNG III. KẾT QUẢ THỰC NGHIỆM CHƯƠNG TRÌNH.....	27
3.1. Dữ liệu thực nghiệm.....	27
3.1.1. <i>EMNIST ByClass</i>	27
3.1.2. <i>EMNIST Balanced</i>	28
3.2. Môi trường thực nghiệm.....	28
3.2.1. <i>Cấu hình phần cứng</i>	28
3.2.2. <i>Cấu hình phần mềm</i>	29
3.3. Kết quả và phân tích.....	29
3.3.1. <i>So sánh kết quả huấn luyện và kiểm thử</i>	29
3.3.2. <i>Đánh giá chương trình ứng dụng</i>	38
KẾT LUẬN.....	44
1. Kết quả đạt được.....	44
2. Hạn chế.....	44
3. Hướng phát triển.....	45
TÀI LIỆU THAM KHẢO.....	46

DANH MỤC HÌNH ẢNH

Hình 1.2.2.1. Sơ đồ CNN trong nhận diện hình ảnh.....	7
Hình 1.2.5.1. Ứng dụng CNN trong an ninh.....	8
Hình 2.1.2.1. Luồng hoạt động.....	12
Hình 2.2.1.1. Pipeline tiền xử lý ảnh đầu vào.....	13
Hình 2.3.1. Cấu trúc mã nguồn.....	18
Hình 2.3.3. Mã nguồn File “config.py”.....	19
Hình 2.3.4. Xử lý dữ liệu và xây dựng pipeline cho dữ liệu.....	20
Hình 2.3.5. Hàm “create_char_cnn_model”.....	20
Hình 2.3.6. Biên dịch mô hình.....	21
Hình 2.3.7. Đánh giá và lưu mô hình.....	21
Hình 2.3.8. Hàm “deskew”.....	22
Hình 2.3.9. Hàm “center_on_mass”.....	22
Hình 2.3.10. Hàm “cleanup_and_standardize_char”.....	23
Hình 2.4.1. Wireframe hoạt động của chương trình.....	24
Hình 3.3.1.1. Ma trận nhầm lẫn với mô hình Byclass.....	30
Hình 3.3.1.2. Ma trận nhầm lẫn với mô hình Balanced.....	31
Hình 3.3.1.3. Báo cáo phân loại của Mô hình Balanced (1).....	31
Hình 3.3.1.4. Báo cáo phân loại của Mô hình Balanced (2).....	32
Hình 3.3.1.5. Báo cáo phân loại của Mô hình Balanced (3).....	32
Hình 3.3.1.6. Báo cáo phân loại của Mô hình Balanced (4).....	33
Hình 3.3.1.7. Báo cáo phân loại của Mô hình Byclass (1).....	34
Hình 3.3.1.8. Báo cáo phân loại của Mô hình Byclass (2).....	35
Hình 3.3.1.9. Báo cáo phân loại của Mô hình Byclass (3).....	36
Hình 3.3.1.10. Báo cáo phân loại của Mô hình Byclass (4).....	36
Hình 3.3.2.1. Nhận dạng chữ “d” viết tay bằng Byclass.....	39
Hình 3.3.2.2. Nhận dạng số “3” viết tay bằng Byclass.....	39
Hình 3.3.2.3. Nhận dạng chữ “E” viết tay bằng Byclass.....	40
Hình 3.3.2.4 Nhận dạng chữ “H” viết tay bằng Byclass.....	40
Hình 3.3.2.5. Nhận dạng chữ “A” viết tay bằng Byclass.....	41
Hình 3.3.2.6. Nhận dạng chữ “o” viết tay bằng Byclass.....	41

Hình 3.3.2.7. Nhận dạng chữ “r” viết tay bằng Byclass.....	42
Hình 3.3.2.8. Nhận dạng chữ “v” viết tay bằng Byclass.....	42
Hình 3.3.2.9. Nhận dạng chữ “O” viết tay bằng Byclass.....	43

DANH MỤC BẢNG BIỂU

Bảng phân công công việc	2
Bảng 3.3.2.1. Testcase chức năng	38

LỜI MỞ ĐẦU

Trong bối cảnh của kỷ nguyên số hóa, việc chuyển đổi dữ liệu từ dạng vật lý sang dạng kỹ thuật số là một nhu cầu thiết yếu, đặc biệt đối với kho tàng thông tin khổng lồ đang được lưu trữ dưới dạng văn bản viết tay. Tuy nhiên, quá trình này gặp phải thách thức lớn do sự đa dạng và không nhất quán trong phong cách viết của mỗi cá nhân, đòi hỏi các giải pháp thông minh và linh hoạt. Để giải quyết vấn đề này, đề tài “Xây dựng chương trình nhận dạng chữ viết dùng mạng nơ ron” được thực hiện nhằm mục tiêu thiết kế và xây dựng một hệ thống phần mềm hoàn chỉnh, có khả năng nhận dạng các ký tự viết tay riêng lẻ từ hình ảnh kỹ thuật số.

Mục tiêu cốt lõi của đề tài là tạo ra một sản phẩm trọn vẹn, không chỉ dừng lại ở việc huấn luyện một mô hình học máy mà còn tích hợp nó vào một ứng dụng thực tiễn có giao diện đồ họa thân thiện. Quá trình này đòi hỏi việc nghiên cứu sâu về kiến trúc mạng nơ ron tích chập (CNN), sau đó áp dụng lý thuyết để huấn luyện một mô hình có độ chính xác cao trên bộ dữ liệu tiêu chuẩn. Song song đó, đề tài tập trung vào việc phát triển một giao diện đồ họa người dùng (GUI) trực quan, nơi các thuật toán tiền xử lý ảnh và mô hình CNN đã huấn luyện được tích hợp một cách liền mạch. Điều này tạo thành một chu trình xử lý khép kín, cho phép người dùng đưa vào một hình ảnh chữ viết tay và nhận lại kết quả là ký tự được nhận dạng cùng với độ tin cậy của dự đoán. Để đảm bảo tính khả thi, phạm vi của đề tài được giới hạn trong việc nhận dạng các ký tự đơn lẻ, được viết tách biệt, và chưa giải quyết bài toán phức tạp hơn là nhận dạng chữ viết tay nối liền.

Ngoài ra, một thách thức không nhỏ trong các bài toán thực tế là sự mất cân bằng về tần suất xuất hiện của các ký tự. Do đó, đề tài cũng sẽ tiến hành thực nghiệm trên hai tập dữ liệu con của EMNIST để so sánh hiệu suất của mô hình trong cả điều kiện lý tưởng và điều kiện mất cân bằng, từ đó rút ra kết luận quan trọng về việc lựa chọn dữ liệu.

Bảng phân công công việc

Thành viên	Công việc	Ghi chú
Nguyễn Quang Hoàng	Xây dựng và huấn luyện mô hình với dữ liệu	Hoàn thành
Phan Đăng Huy	Kiểm thử	Hoàn thành
Ngô Văn Quân	Xây dựng giao diện người dùng	Hoàn thành
Đào Văn Thái	Xây dựng thuật toán tiền xử lí ảnh	Hoàn thành
Hoàng Thanh Tùng	Trình bày cơ sở lý thuyết	Hoàn thành

CHƯƠNG I. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan về trí tuệ nhân tạo

1.1.1. Lịch sử phát triển của trí tuệ nhân tạo

Trí tuệ nhân tạo là một lĩnh vực của khoa học máy tính nhằm tạo ra các cỗ máy thông minh có khả năng thực hiện các nhiệm vụ đòi hỏi trí thông minh của con người. Lịch sử của nó là một hành trình đầy những đột phá và cả những thách thức:

- Giai đoạn nền móng (1943-1956): Những ý tưởng đầu tiên về mạng nơ ron nhân tạo và khả năng "suy nghĩ" của máy móc được hình thành. Cột mốc quan trọng nhất là Hội thảo Dartmouth năm 1956, nơi thuật ngữ "Trí tuệ nhân tạo" được khai sinh, chính thức đặt nền móng cho lĩnh vực này.
- Thời kỳ lạc quan (1956-1974): Giai đoạn chứng kiến sự phát triển ban đầu với nhiều kỳ vọng lớn lao, các chương trình AI có thể chơi cờ, giải toán và chứng minh định lý.
- "Mùa đông AI" (1974-1980 và 1987-1993): Do những hạn chế về sức mạnh tính toán và sự phức tạp của các bài toán thực tế, AI đã không thể đáp ứng được những kỳ vọng ban đầu. Điều này dẫn đến việc cắt giảm tài trợ và làm chậm quá trình nghiên cứu.
- Sự trỗi dậy của hệ chuyên gia (Thập niên 80): AI tìm thấy thành công thương mại đầu tiên với các hệ chuyên gia, mô phỏng kiến thức của con người trong một lĩnh vực hẹp để đưa ra quyết định.
- Cách mạng học máy và học sâu (1990-nay): Với sự bùng nổ của dữ liệu lớn (Big Data) và sức mạnh xử lý của phần cứng (GPU), các phương pháp Học máy (Machine Learning) và đặc biệt là Học sâu (Deep Learning) đã tạo ra những đột phá ngoạn mục, đưa AI trở thành một công nghệ cốt lõi trong cuộc sống hiện đại.

1.1.2. Các hướng nghiên cứu chính trong trí tuệ nhân tạo

AI là một lĩnh vực rộng lớn bao gồm nhiều nhánh nghiên cứu chuyên biệt:

- Học máy (Machine Learning): Là phương pháp cho phép máy tính "học" từ dữ liệu để đưa ra dự đoán hoặc quyết định mà không cần lập trình tường minh.

- Học sâu (Deep Learning): Là một tập hợp con của học máy, sử dụng mạng nơ ron nhiều lớp để tự động học các đặc trưng phức tạp từ lượng lớn dữ liệu. Đây chính là hướng tiếp cận được sử dụng trong đề tài này.
- Thị giác máy tính (Computer Vision): Nghiên cứu cách máy tính có thể "nhìn" và diễn giải thông tin từ hình ảnh và video. Nhận dạng chữ viết tay là một bài toán kinh điển trong lĩnh vực này.
- Xử lý ngôn ngữ tự nhiên (Natural Language Processing - NLP): Giúp máy tính có khả năng hiểu, diễn giải và tạo ra ngôn ngữ của con người.
- Robotics: Tích hợp AI để robot có thể nhận thức môi trường và thực hiện các hành động một cách thông minh.

1.1.3. Các ứng dụng tiêu biểu của trí tuệ nhân tạo

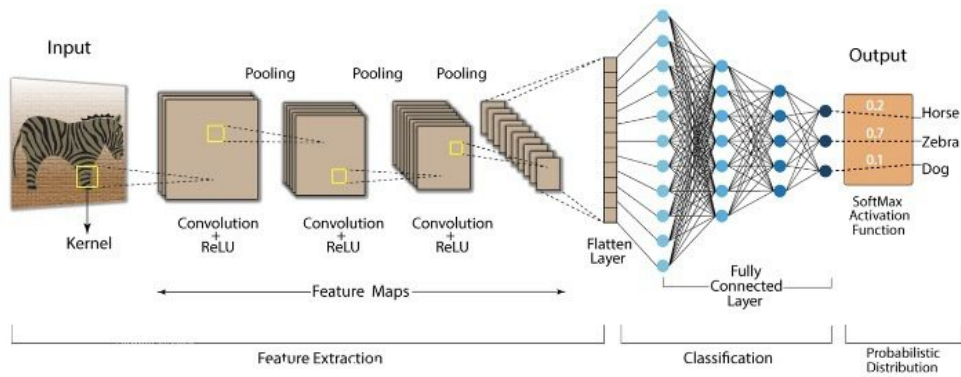
Trí tuệ nhân tạo đã và đang được ứng dụng rộng rãi, làm thay đổi sâu sắc nhiều ngành công nghiệp:

- Y tế: Hỗ trợ chẩn đoán bệnh qua hình ảnh y khoa, phát hiện thuốc mới, cá nhân hóa phác đồ điều trị.
- Tài chính: Phát hiện gian lận giao dịch, phân tích rủi ro tín dụng, giao dịch tự động.
- Giao thông: Xe tự hành, tối ưu hóa luồng giao thông, quản lý chuỗi cung ứng.
- Thương mại điện tử: Hệ thống gợi ý sản phẩm, chatbot hỗ trợ khách hàng, tối ưu kho.
- Giải trí: Trợ lý ảo, hệ thống đề xuất phim/nhạc, tạo nhân vật thông minh trong game.

1.2. Mạng nơ ron tích chập (CNN)

1.2.1. Kiến trúc cơ bản của CNN

Một CNN thường được xây dựng từ nhiều lớp xếp chồng lên nhau. Các lớp này phối hợp với nhau để trích xuất đặc trưng và thực hiện phân loại hoặc dự đoán. Điểm cốt lõi của kiến trúc này là khả năng học các đặc trưng theo từng cấp độ: các lớp đầu tiên nhận diện những chi tiết đơn giản như cạnh và góc, sau đó các lớp sâu hơn sẽ tổng hợp chúng thành những hình dạng phức tạp hơn để đưa ra quyết định cuối cùng.



Hình 1.2.1.1. Sơ đồ kiến trúc của 1 Mạng nơ-ron tích chập

a, Lớp tích chập (Convolutional Layer)

Đây là lớp quan trọng nhất trong CNN. Ý tưởng là sử dụng các bộ lọc (kernel) kích thước nhỏ, ví dụ 3×3 hoặc 5×5 , trượt qua toàn bộ ảnh đầu vào. Mỗi bộ lọc sẽ tạo ra một bản đồ đặc trưng thể hiện những chi tiết quan trọng như cạnh, góc, hoa văn.

Ví dụ: ảnh đầu vào kích thước $32 \times 32 \times 3$ (3 kênh màu RGB). Nếu sử dụng 6 bộ lọc 5×5 , ta sẽ thu được 6 bản đồ đặc trưng, mỗi bản đồ có kích thước nhỏ hơn ảnh gốc. Ở các lớp đầu tiên, mạng học được các đặc trưng đơn giản (cạnh, góc), còn ở lớp sâu hơn, đặc trưng trở nên phức tạp hơn (mắt, mũi, khuôn mặt).

b, Lớp kích hoạt phi tuyến (Activation Layer)

Sau khi tích chập, các giá trị đầu ra sẽ được đưa qua một hàm kích hoạt để tăng khả năng biểu diễn. Hàm kích hoạt thường dùng là ReLU, giúp mạng bỏ qua giá trị không quan trọng và giữ lại đặc trưng chính. Nhờ đó, CNN có thể học được mối quan hệ phức tạp trong dữ liệu thay vì chỉ tuyến tính đơn giản.

c, Lớp gộp (Pooling Layer)

Lớp gộp giúp giảm kích thước của bản đồ đặc trưng, giữ lại thông tin quan trọng nhất. Thay vì giữ nguyên toàn bộ chi tiết, lớp gộp chỉ chọn ra đặc trưng nổi bật trong mỗi vùng nhỏ.

Ví dụ: nếu bản đồ đặc trưng có kích thước 28×28 , sau khi gộp với cửa sổ 2×2 , kích thước giảm còn 14×14 . Nhờ vậy, mô hình nhẹ hơn, ít tham số hơn, học nhanh hơn và bền vững hơn với các thay đổi nhỏ trong ảnh (chẳng hạn vật thể bị dịch chuyển một chút).

d, Lớp kết nối đầy đủ (Fully Connected Layer – FC)

Sau nhiều tầng tích chập và gộp, ảnh ban đầu được nén lại thành một vector đặc trưng. Lớp kết nối đầy đủ có nhiệm vụ tổng hợp tất cả các đặc trưng này để đưa ra dự đoán cuối cùng.

Ví dụ: một vector đặc trưng 400 chiều có thể được đưa qua một lớp kết nối với 120 neuron, sau đó là 84 neuron, và cuối cùng là đầu ra.

e, Lớp đầu ra (Output Layer)

Lớp đầu ra tùy thuộc vào bài toán. Nếu là phân loại hình ảnh, lớp này sẽ đưa ra xác suất ảnh thuộc về từng lớp. Nếu là hồi quy, nó có thể xuất ra một giá trị duy nhất, chẳng hạn như dự đoán tuổi hoặc giá trị số.

f, Minh họa dòng dữ liệu qua CNN

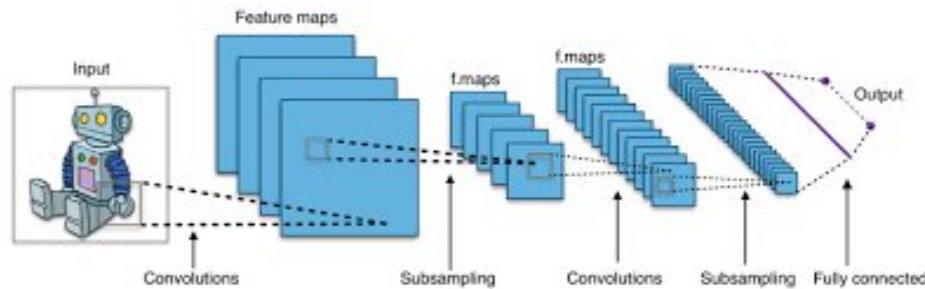
Ví dụ với ảnh màu $32 \times 32 \times 3$ (dữ liệu CIFAR-10): Ảnh đầu vào ($32 \times 32 \times 3$) $\rightarrow$ Lớp tích chập + kích hoạt ($28 \times 28 \times 6$) $\rightarrow$ Lớp gộp ($14 \times 14 \times 6$) $\rightarrow$ Lớp tích chập + kích hoạt ($10 \times 10 \times 16$) $\rightarrow$ Lớp gộp ($5 \times 5 \times 16$) $\rightarrow$ Vector đặc trưng (400 chiều) $\rightarrow$ Các lớp kết nối đầy đủ ($120 \rightarrow 84$ neuron) $\rightarrow$ Lớp đầu ra (10 lớp).

1.2.2. Quá trình huấn luyện CNN

Trong quá trình huấn luyện một mạng nơ ron, mô hình học hỏi thông qua một chu trình gồm hai giai đoạn chính:

- Lan truyền thuận (Forward Propagation), nơi dữ liệu đầu vào, chẳng hạn như một hình ảnh, sẽ được đưa qua một chuỗi các lớp của mạng. Mỗi lớp sẽ thực hiện các phép biến đổi toán học để trích xuất các đặc trưng ngày càng phức tạp từ các cạnh và góc đơn giản ở lớp đầu tiên đến các hình dạng và cấu trúc tinh vi hơn ở các lớp sâu hơn, và cuối cùng đưa ra một dự đoán ban đầu.
- Lan truyền ngược (Backpropagation): Giai đoạn này bắt đầu bằng việc tính toán sai số (loss) giữa dự đoán và nhãn thật qua một hàm mất mát (loss function). Dựa trên sai số này, thuật toán lan truyền ngược sẽ tính gradient (đạo hàm) của hàm mất mát theo từng trọng số, về bản chất là xác định mức độ "góp phần" vào sai số của mỗi trọng số.

Toàn bộ chu trình lan truyền thuận và lan truyền ngược này không chỉ diễn ra một lần. Quá trình này được thực hiện qua nhiều lần lặp (iterations) và nhiều chu kỳ (epochs), mỗi epoch tương ứng với một lần duyệt qua toàn bộ tập dữ liệu huấn luyện. Qua mỗi lần lặp, mô hình lại "học hỏi" từ lỗi sai của mình và tinh chỉnh các trọng số để trở nên chính xác hơn, cho đến khi hiệu suất của mô hình đạt đến mức tối ưu và hội tụ.



Hình 1.2.2.1. Sơ đồ CNN trong nhận diện hình ảnh

1.2.3. Ưu điểm của CNN

CNN mang nhiều ưu điểm đột phá, khiến nó trở thành một trong những mô hình học sâu hiệu quả và phổ biến nhất hiện nay. Trước hết, CNN có khả năng tự động học các đặc trưng từ dữ liệu đầu vào, nhờ đó không cần thiết kể thủ công các đặc trưng như trong các phương pháp Machine Learning truyền thống.

Nhờ cơ chế chia sẻ trọng số và kết nối cục bộ, CNN sử dụng ít tham số hơn so với mạng nơ ron truyền thống (MLP), giúp mô hình gọn nhẹ và huấn luyện nhanh hơn. CNN cũng thể hiện hiệu quả vượt trội trong các bài toán xử lý ảnh, video và dữ liệu chuỗi nhờ khả năng phát hiện và trích xuất các mẫu phức tạp. Ngoài ra, CNN có khả năng tổng quát hóa tốt, phát hiện được các đặc trưng ở nhiều mức trừu tượng khác nhau, từ các cạnh và hình dạng cơ bản đến các đối tượng phức tạp, nhờ đó trở thành công cụ quan trọng trong nhiều ứng dụng thực tiễn như thị giác máy tính, y tế, giao thông thông minh và giám sát an ninh.

1.2.4. Nhược điểm của CNN

Dù sở hữu nhiều ưu điểm vượt trội, CNN vẫn tồn tại một số nhược điểm nhất định cần lưu ý. Trước hết, mô hình yêu cầu lượng dữ liệu huấn luyện lớn; nếu dữ liệu không đủ, mạng dễ bị overfitting, dẫn đến giảm khả năng tổng quát hóa.

Chi phí tính toán của CNN khá cao do cần phần cứng mạnh như GPU hoặc TPU để huấn luyện và triển khai mô hình hiệu quả. Một hạn chế khác là tính “black-box” của CNN, khiến việc giải thích lý do tại sao mô hình đưa ra dự đoán cụ thể trở nên khó khăn, điều này có thể gây trở ngại trong các ứng dụng yêu cầu minh bạch cao. Ngoài ra, CNN còn nhạy cảm với các biến dạng dữ liệu đặc biệt, chẳng hạn khi ảnh bị xoay lớn, thay đổi tỉ lệ hoặc biến dạng mạnh, điều này có thể ảnh hưởng đáng kể đến hiệu suất và độ chính xác của mô hình.

1.2.5. Ứng dụng của CNN

Hiện nay, Mạng Nơ ron Tích chập (CNN) được ứng dụng rộng rãi trong nhiều lĩnh vực nhờ khả năng trích xuất đặc trưng hiệu quả và xử lý dữ liệu phức tạp. Trong lĩnh vực cốt lõi là thị giác máy tính, CNN là nền tảng cho các tác vụ như nhận dạng ảnh, phát hiện đối tượng và phân đoạn ảnh với độ chính xác cao.

Năng lực này đã được mở rộng sang nhiều ngành công nghiệp quan trọng: trong y tế, mô hình được sử dụng để phân tích ảnh X-quang, MRI và CT, giúp các bác sĩ phát hiện và chẩn đoán bệnh lý một cách nhanh chóng và chính xác. Đối với lĩnh vực ô tô tự lái, CNN đóng vai trò thiết yếu trong việc nhận diện làn đường, chướng ngại vật và người đi bộ, góp phần trực tiếp nâng cao an toàn giao thông. Tương tự, trong lĩnh vực an ninh, công nghệ này hỗ trợ các hệ thống nhận diện khuôn mặt và giám sát qua camera, giúp cải thiện hiệu quả quản lý và kiểm soát an ninh. Ngoài ra, vượt ra ngoài phạm vi hình ảnh, CNN còn được áp dụng hiệu quả trong xử lý ngôn ngữ tự nhiên để trích xuất đặc trưng văn bản, phân tích cảm xúc và hỗ trợ nhiều tác vụ liên quan đến dữ liệu chuỗi.



Hình 1.2.5.1. Ứng dụng CNN trong an ninh

1.3. Các công cụ, thư viện được sử dụng trong đề tài

1.3.1. TensorFlow và Keras

- Mô tả: TensorFlow là nền tảng học máy mã nguồn mở hàng đầu, được phát triển bởi Google. Keras là một API bậc cao chạy trên nền TensorFlow, giúp đơn giản hóa việc xây dựng và huấn luyện các mô hình mạng nơ ron.
- Ứng dụng trong đề tài: Đây là công cụ cốt lõi được sử dụng để xây dựng, huấn luyện và kiểm thử mô hình Mạng Nơ ron Tích chập (CNN). Keras cung cấp các lớp (Layer) có sẵn giúp việc thiết kế kiến trúc mô hình trở nên trực quan và nhanh chóng.

1.3.2. TensorFlow Datasets

- Mô tả: Là một thư viện cung cấp quyền truy cập vào hàng trăm bộ dữ liệu sẵn sàng sử dụng, bao gồm cả EMNIST.
- Ứng dụng trong đề tài: Thư viện này được dùng để tải và chuẩn bị hai bộ dữ liệu EMNIST (byclass và balanced) một cách thuận tiện, đảm bảo tính toàn vẹn và tiêu chuẩn của dữ liệu đầu vào cho quá trình huấn luyện.

1.3.3. NumPy

- Mô tả: Là thư viện nền tảng cho tính toán khoa học trong Python, cung cấp các cấu trúc dữ liệu mảng đa chiều hiệu suất cao.
- Ứng dụng trong đề tài: NumPy được sử dụng rộng rãi để xử lý dữ liệu dạng số, đặc biệt là các mảng (array) biểu diễn hình ảnh. Các thao tác như chuẩn hóa giá trị pixel, thay đổi chiều dữ liệu (reshape) để phù hợp với đầu vào của mô hình TensorFlow đều được thực hiện bằng NumPy.

1.3.4. OpenCV

- Mô tả: Là thư viện hàng đầu về thị giác máy tính, cung cấp vô số thuật toán và công cụ để xử lý hình ảnh và video.
- Ứng dụng trong đề tài: OpenCV đóng vai trò quan trọng trong việc tiền xử lý hình ảnh đầu vào trong ứng dụng GUI. Các chức năng như đọc file ảnh, chuyển đổi không gian màu, thay đổi kích thước, và các kỹ thuật xử lý ảnh nâng cao trước khi đưa vào dự đoán đều do OpenCV đảm nhiệm.

1.3.5. Pillow

- Mô tả: Là thư viện xử lý ảnh mạnh mẽ, kế thừa từ thư viện PIL (Python Imaging Library) cũ.
- Ứng dụng trong đề tài: Pillow đóng vai trò là cầu nối giữa định dạng ảnh của OpenCV (mảng NumPy) và định dạng ảnh của Tkinter. Nó được sử dụng để chuyển đổi hình ảnh chúng có thể được hiển thị trực tiếp trên giao diện đồ họa.

1.3.6. Tkinter

- Mô tả: Là thư viện GUI (Giao diện đồ họa người dùng) tiêu chuẩn của Python.
- Ứng dụng trong đề tài: Toàn bộ giao diện đồ họa của chương trình bao gồm cửa sổ, nút bấm, khung hiển thị ảnh và kết quả đều được xây dựng bằng Tkinter.

1.3.7. Matplotlib và Seaborn

- Mô tả: Matplotlib là thư viện vẽ đồ thị 2D phổ biến, trong khi Seaborn là một thư viện dựa trên Matplotlib giúp tạo ra các đồ thị thống kê đẹp mắt hơn.
- Ứng dụng trong đề tài: Bộ đôi này được sử dụng trong giai đoạn đánh giá mô hình để trực quan hóa kết quả. Cụ thể, Seaborn được dùng để vẽ ma trận nhầm lẫn (Confusion Matrix) dưới dạng heatmap, giúp đánh giá hiệu suất của mô hình một cách trực quan và chi tiết.

CHƯƠNG II. XÂY DỰNG CHƯƠNG TRÌNH

2.1. Mô tả bài toán

2.1.1. Phân tích yêu cầu hệ thống

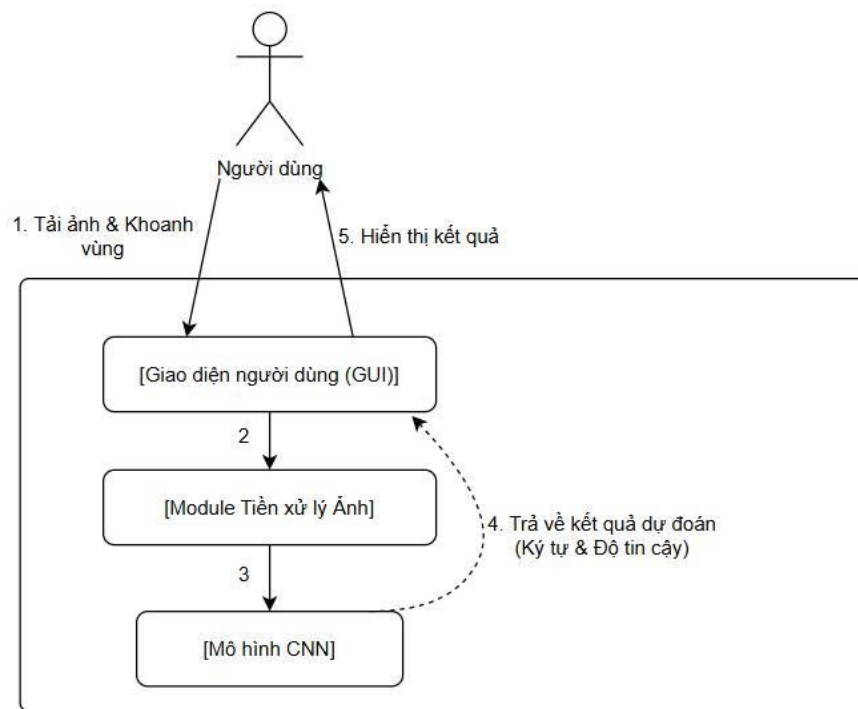
Bài toán nhận dạng ký tự viết tay là một vấn đề cốt lõi trong lĩnh vực nhận dạng mẫu (Pattern Recognition) và thị giác máy tính (Computer Vision). Từ góc độ Trí tuệ Nhân tạo, bài toán này đặt ra thách thức làm thế nào để xây dựng một hệ thống có khả năng "nhìn" và diễn giải một hình ảnh, sau đó ánh xạ thông tin thị giác đó vào một lớp khái niệm trừu tượng (ký tự cụ thể). Khó khăn cơ bản của bài toán không nằm ở việc xử lý hình ảnh đơn thuần, mà ở việc xây dựng một mô hình có khả năng tổng quát hóa trước sự biến thiên gần như vô hạn của dữ liệu đầu vào do mỗi người có một phong cách viết tay riêng, với sự khác biệt về nét chữ, kích thước, độ nghiêng và cả các biến dạng nhỏ.

Các phương pháp xử lý ảnh truyền thống dựa trên các quy tắc được định nghĩa trước (rule-based) hoặc các thuật toán trích xuất đặc trưng thủ công tỏ ra kém hiệu quả và không đủ linh hoạt để đối phó với sự đa dạng này. Do đó, bài toán đòi hỏi một cách tiếp cận dựa trên dữ liệu (data-driven approach), nơi một mô hình có thể tự động học các đặc trưng nhận dạng trực tiếp từ các ví dụ. Trong khuôn khổ đề tài này, bài toán được giải quyết bằng việc ứng dụng “Mạng nơ ron tích chập (CNN)”, một kiến trúc học sâu được thiết kế đặc biệt để khai thác cấu trúc không gian của dữ liệu hình ảnh. Cách tiếp cận này giải quyết bài toán trên cơ sở sau:

- Biểu diễn dữ liệu: Mỗi hình ảnh ký tự được biểu diễn dưới dạng một ma trận pixel. Bài toán được mô hình hóa thành việc xây dựng một hàm phức tạp, có khả năng ánh xạ từ không gian ma trận pixel đầu vào (ví dụ: 28x28 chiều) đến một không gian đầu ra gồm N lớp (với N là tổng số ký tự cần nhận dạng).
- Học đặc trưng tự động: Thay vì trích xuất đặc trưng thủ công, mô hình CNN được sử dụng để tự động học một hệ thống phân cấp các đặc trưng.
- Các lớp tích chập (Convolutional Layers) ở tầng sâu dưới sẽ học cách nhận diện các đặc trưng nguyên thủy và cục bộ như các cạnh thẳng, đường cong, góc nhọn từ ma trận pixel. Khi đi sâu hơn qua các lớp mạng, các đặc trưng đơn giản này sẽ được tổng hợp lại để hình thành các đặc trưng phức tạp và trừu tượng hơn.

- Phân loại dựa trên đặc trưng: Sau khi các đặc trưng bậc cao đã được trích xuất, các lớp kết nối đầy đủ (Fully Connected Layers) ở cuối mạng sẽ đóng vai trò như một bộ phân loại thông minh. Nó nhận đầu vào là vector đặc trưng và đưa ra quyết định cuối cùng, ánh xạ các đặc trưng đó vào lớp ký tự tương ứng.

2.1.2. Thiết kế kiến trúc tổng quan



Hình 2.1.2.1. Luồng hoạt động

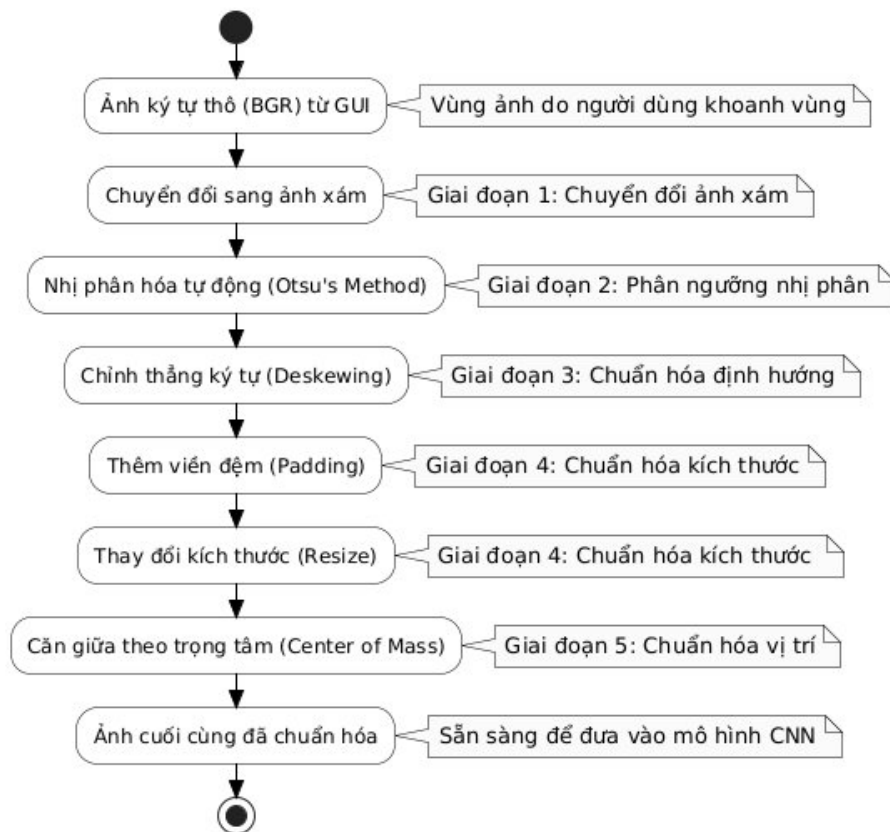
Hệ thống được cấu thành từ 3 module chính hoạt động bên trong một ứng dụng thống nhất:

- Giao diện người dùng: Nhiệm vụ của GUI không chỉ là hiển thị hình ảnh và kết quả, mà còn chịu trách nhiệm thu nhận các sự kiện đầu vào như nhấn nút "Tải ảnh", khoanh vùng ký tự bằng chuột, và kích hoạt quy trình nhận dạng.
- Module tiền xử lý ảnh: Khi nhận một vùng ảnh ký tự thô do người dùng cắt, module này sẽ tự động thực hiện một chuỗi các tác vụ: chuyển đổi sang ảnh xám, nhị phân hóa bằng thuật toán Otsu để tách biệt ký tự và nền, chỉnh thẳng ký tự bị nghiêng (deskew), và cuối cùng là căn chỉnh ký tự vào chính giữa một khung ảnh có kích thước tiêu chuẩn là 28x28 pixel.

2.2. Mô tả chi tiết giải thuật

2.2.1. Pipeline tiền xử lý ảnh đầu vào

Để đảm bảo tính nhất quán của dữ liệu đầu vào và tối ưu hóa hiệu suất của mô hình mạng nơ ron tích chập, một pipeline tiền xử lý đa giai đoạn đã được xây dựng và triển khai. Mục tiêu của pipeline này là thực hiện các phép chuẩn hóa về mặt quang học và hình học đối với ảnh ký tự thô do người dùng cung cấp. Quá trình này có vai trò triệt tiêu các biến số ngoại lai (ví dụ: độ nghiêng, vị trí, kích thước, điều kiện ánh sáng), từ đó tạo ra một không gian đặc trưng đồng nhất, cho phép mô hình tập trung vào việc học các thuộc tính hình thái bất biến của ký tự để phục vụ cho việc phân loại.



Hình 2.2.1.1. Pipeline tiền xử lý ảnh đầu vào

a, Giai đoạn 1: Chuyển đổi sang không gian màu xám

- Mục tiêu: Giảm độ phức tạp tính toán và số chiều của dữ liệu đầu vào. Thông tin màu sắc, được biểu diễn qua ba kênh màu (BGR), được xem là không mang lại giá trị định danh cho bài toán phân loại hình dạng ký tự và do đó có thể được loại bỏ.

- Quy trình kỹ thuật: Sử dụng hàm `cv2.cvtColor()` của thư viện OpenCV để thực hiện phép biến đổi từ không gian màu BGR sang thang độ xám. Phép biến đổi này áp dụng một công thức trung bình có trọng số lên giá trị của ba kênh màu để tính toán ra một giá trị cường độ sáng duy nhất cho mỗi pixel.
- Kết quả: Dữ liệu ảnh được chuyển đổi từ một tensor 3 chiều thành một ma trận 2 chiều, giúp giảm hơn 66% khối lượng dữ liệu cần xử lý và tối ưu hóa hiệu suất cho các giai đoạn tiếp theo.

b, Giai đoạn 2: Phân ngưỡng nhị phân bằng thuật toán Otsu

- Mục tiêu: Thực hiện phân đoạn (segmentation) hình ảnh để tách biệt hoàn toàn đối tượng ký tự (foreground) khỏi nền (background). Quá trình này chuyển đổi ảnh xám thành ảnh nhị phân, loại bỏ các biến thể về độ sáng và độ tương phản.
- Quy trình kỹ thuật: Để giải quyết hạn chế của việc sử dụng một ngưỡng cố định, hệ thống triển khai phương pháp của Otsu thông qua cờ `cv2.THRESH_OTSU` trong hàm `cv2.threshold()`. Thuật toán này hoạt động bằng cách phân tích biểu đồ tần suất (histogram) của ảnh để tự động xác định một giá trị ngưỡng tối ưu trên toàn cục. Ngưỡng này được xác định tại điểm mà phương sai nội lớp (intra-class variance) của hai nhóm pixel (foreground và background) đạt giá trị cực tiểu, qua đó tối đa hóa phương sai giữa hai lớp. Cờ `cv2.THRESH_BINARY_INV` được sử dụng đồng thời để đảm bảo ký tự được biểu diễn bằng pixel trắng (giá trị 255) và nền bằng pixel đen (giá trị 0).
- Kết quả: Tạo ra một ảnh nhị phân có độ tương phản cao, định nghĩa rõ ràng hình thái của ký tự và cung cấp một đầu vào ổn định cho các thuật toán phân tích hình học.

c, Giai đoạn 3: Chuẩn hóa định hướng bằng thuật toán chỉnh thẳng

- Mục tiêu: Hiệu chỉnh và chuẩn hóa độ nghiêng của ký tự, một biến số hình học phổ biến gây ra bởi sự khác biệt trong phong cách viết tay của mỗi cá nhân.
- Quy trình kỹ thuật: Thuật toán này dựa trên việc phân tích mô-men hình ảnh (Image Moments). Cụ thể, các mô-men trung tâm bậc hai (μ_{02} và μ_{11}), vốn bất biến với phép tịnh tiến, được tính toán để xác định phương sai và hiệp phương sai của phân bố pixel, từ đó suy ra định hướng của đối tượng.
- Kết quả: Loại bỏ sự biến thiên về góc viết, đảm bảo các ký tự cùng lớp có định hướng hình học tương đồng trước khi đưa vào mô hình.

d, Giai đoạn 4: Chuẩn hóa kích thước

- Mục tiêu: Đảm bảo tính đồng nhất về kích thước của toàn bộ dữ liệu đầu vào, đưa ảnh về kích thước cố định 28×28 pixel theo yêu cầu của kiến trúc mạng CNN.
- Quy trình kỹ thuật: Quy trình bao gồm hai bước nhỏ. Đầu tiên, một đường viền đệm màu đen có độ dày không đổi được thêm vào xung quanh ảnh ký tự (Padding) bằng hàm `cv2.copyMakeBorder`. Thao tác này nhằm mục đích tạo một vùng an toàn để ngăn chặn việc mất mát thông tin tại các pixel rìa trong quá trình co giãn ảnh. Tiếp theo, ảnh đã được đệm viền sẽ được thay đổi kích thước bằng hàm `cv2.resize` với phương pháp nội suy `cv2.INTER_AREA`.
- Kết quả: Tạo ra một ma trận pixel có kích thước 28×28 , phù hợp với yêu cầu của lớp đầu vào trong mô hình CNN.

e, Giai đoạn 5: Chuẩn hóa vị trí bằng căn giữa theo trọng tâm

- Mục tiêu: Chuẩn hóa vị trí không gian của ký tự trong khung ảnh 28×28 , đảm bảo đối tượng luôn được định vị tại trung tâm hình học của ảnh.
- Quy trình kỹ thuật: Bước này sử dụng mô-men hình ảnh. Sau khi xác định được trọng tâm, một vector dịch chuyển (`shiftx`, `shifty`) được tính toán để di dời điểm này về trung tâm của khung ảnh (tọa độ 14,14). Một ma trận biến đổi affine chỉ chứa thông tin dịch chuyển (translation) được áp dụng để thực hiện việc tái định vị này.
- Kết quả: Ảnh đầu ra là một biểu diễn ký tự đã được chuẩn hóa toàn diện về mặt quang học và hình học, sẵn sàng cho quá trình trích xuất đặc trưng và phân loại của mô hình.

2.2.2. Kiến trúc mô hình CNN và huấn luyện mô hình

Đây là giai đoạn cốt lõi, nơi mô hình học máy được dạy để nhận biết và phân loại các ký tự. Mục tiêu của giai đoạn này là tìm ra một tập hợp trọng số tối ưu cho mạng nơ ron sao cho hàm mất mát (đại diện cho sai số) trên dữ liệu là nhỏ nhất. Kiến trúc này bao gồm hai thành phần chính:

- Phần trích xuất đặc trưng:

+ Khối 1: Gồm hai lớp Conv2D với 32 bộ lọc (kích thước 3×3 , hàm kích hoạt ReLU), theo sau là một lớp BatchNormalization. Khối này kết thúc bằng một lớp MaxPooling2D để giảm kích thước và một lớp Dropout (tỷ lệ 0.25) để chính quy hóa.

+ Khối 2: Gồm hai lớp Conv2D với số bộ lọc tăng lên 64, với BatchNormalization, MaxPooling2D, và Dropout (tỷ lệ 0.25). Việc tăng số bộ lọc ở các lớp sâu hơn giúp mô hình học các đặc trưng phức tạp hơn.

- Phần Phân loại:

+ Sau khi các đặc trưng được trích xuất và làm phẳng bởi lớp Flatten, chúng được đưa vào một lớp Dense với 512 nơ ron (kích hoạt ReLU), kèm theo BatchNormalization và một lớp Dropout với tỷ lệ cao (0.5) để chống quá khớp mạnh mẽ ở giai đoạn cuối.

+ Lớp Dense đầu ra sử dụng hàm kích hoạt softmax và có số nơ ron bằng với số lớp của bộ dữ liệu (NUM_CLASSES = 62 cho ByClass, 47 cho Balanced), trả về một vector xác suất cho việc phân loại.

Việc sử dụng các lớp BatchNormalization sau mỗi khối tích chập và lớp Dense giúp ổn định quá trình huấn luyện và tăng tốc độ hội tụ. Các lớp Dropout được thêm vào ở các vị trí chiến lược để giảm thiểu sự phụ thuộc lẫn nhau giữa các nơ ron, từ đó nâng cao khả năng tổng quát hóa của mô hình.

a, Chuẩn bị và tiền xử lý dữ liệu

Đầu tiên, bộ dữ liệu EMNIST được tải về bằng thư viện “tensorflow-datasets” để đảm bảo tính nhất quán. Sau đó, một pipeline dữ liệu được xây dựng với “tf.data” để tối ưu hóa việc nạp dữ liệu. Thay vì tải toàn bộ hàng trăm nghìn ảnh vào RAM, pipeline này đọc và xử lý dữ liệu hiệu quả theo từng lô nhỏ, giúp tiết kiệm tài nguyên.

Trên mỗi hình ảnh trong pipeline, một chuỗi các phép biến đổi được áp dụng. Giá trị pixel của ảnh, vốn có thang độ từ 0 đến 255, được chuẩn hóa về khoảng [0, 1]. Đây là một bước cực kỳ quan trọng, giúp ổn định gradient trong quá trình lan truyền ngược và tăng tốc độ hội tụ của mô hình. Nhãn của ảnh (ví dụ, số 10 đại diện cho ký tự 'A') được chuyển đổi sang định dạng one-hot (một vector nhị phân, ví dụ [0,0,...,1,...,0]). Định dạng này loại bỏ mối quan hệ thứ tự giả tạo giữa các lớp và là yêu cầu đầu vào cho hàm mất mát.

b, Khởi tạo kiến trúc mạng nơ ron

Một kiến trúc mạng CNN được khởi tạo, bao gồm nhiều lớp được xếp chồng lên nhau. Các trọng số ban đầu của mạng được gán các giá trị ngẫu nhiên. Kiến trúc này được chia làm hai phần chính:

- Phần Trích xuất đặc trưng: Bao gồm các khối Lớp Tích chập và Lớp Gộp xen kẽ. Nhiệm vụ của phần này là tự động học cách nhận diện các mẫu hình ảnh. Các lớp đầu tiên sẽ học các đặc trưng đơn giản như cạnh và góc, trong khi các lớp sâu hơn sẽ kết hợp chúng để nhận diện các cấu trúc phức tạp hơn như đường cong, vòng lặp, đặc trưng cho từng ký tự.
- Phần phân loại: Bao gồm các lớp kết nối Đầy đủ. Sau khi các đặc trưng bậc cao được trích xuất, phần này có nhiệm vụ tổng hợp chúng và đưa ra quyết định phân loại cuối cùng.

c, Thực hiện vòng lặp huấn luyện

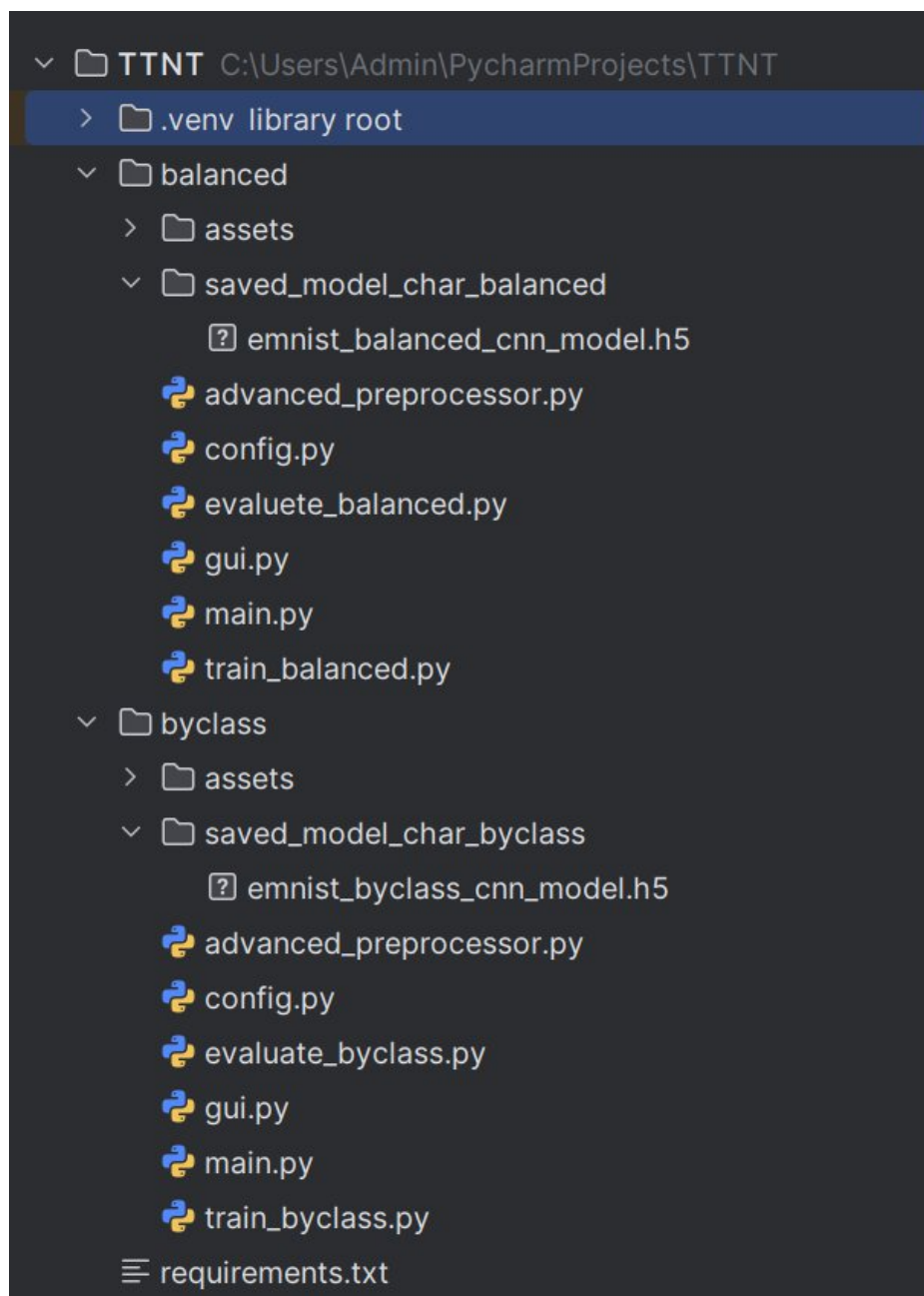
Quá trình huấn luyện là một vòng lặp qua nhiều chu kỳ (epoch). Trong mỗi chu kỳ, thuật toán thực hiện các bước sau:

- Lan truyền thuận (Forward Propagation): Một lô (batch) các hình ảnh từ bộ dữ liệu được đưa vào mạng. Dữ liệu đi qua từng lớp, từ đầu vào đến đầu ra, để tạo ra một dự đoán ban đầu cho mỗi ảnh.
- Tính toán sai số (Loss Calculation): Dự đoán của mô hình được so sánh với nhãn thật. Một hàm mất mát (loss function) sẽ tính toán một giá trị đại diện cho mức độ sai lệch giữa dự đoán và thực tế.
- Lan truyền ngược (Backpropagation) và cập nhật trọng số: Sai số này sau đó được lan truyền ngược lại qua mạng. Dựa trên mức độ "đóng góp" vào sai số của từng trọng số, một thuật toán tối ưu sẽ thực hiện một điều chỉnh nhỏ trên toàn bộ các trọng số của mạng. Mục đích của việc điều chỉnh này là để trong lần lan truyền thuận tiếp theo, dự đoán của mô hình sẽ gần với nhãn thật hơn.

Vòng lặp này được lặp lại cho đến khi sai số của mô hình trên một tập dữ liệu kiểm định riêng biệt không còn giảm nữa, lúc này mô hình được coi là đã "hội tụ" và quá trình huấn luyện kết thúc. Mô hình với các trọng số tốt nhất sẽ được lưu lại.

2.3. Cấu trúc mã nguồn

Dự án được tổ chức thành hai thư mục cấp cao là /byclass và /balanced. Cấu trúc này cho phép chứa hai phiên bản hoàn chỉnh và độc lập của ứng dụng, mỗi phiên bản tương ứng với một mô hình đã được huấn luyện trên một bộ dữ liệu khác nhau. Mỗi thư mục con chứa một bộ mã nguồn, tài sản (assets) và model đã lưu của riêng nó, đảm bảo không có sự xung đột và cho phép kiểm thử, triển khai từng phiên bản một cách riêng biệt.



Hình 2.3.1. Cấu trúc mã nguồn

a, File “config.py”

```
1  # config.py
2  # Đường dẫn đến file model đã huấn luyện
3  MODEL_PATH = 'saved_model_char_balanced/emnist_balanced_cnn_model.h5'
4
5  # Cấu hình ảnh
6  IMG_SIZE = 28
7  CHANNELS = 1
8
9  # Bộ ký tự
10 NUM_CLASSES = 47
11 LABEL_MAPPING = {
12     0: '0', 1: '1', 2: '2', 3: '3', 4: '4', 5: '5', 6: '6', 7: '7', 8: '8', 9: '9',
13     10: 'A', 11: 'B', 12: 'C', 13: 'D', 14: 'E', 15: 'F', 16: 'G', 17: 'H', 18: 'I',
14     19: 'J', 20: 'K', 21: 'L', 22: 'M', 23: 'N', 24: 'O', 25: 'P', 26: 'Q', 27: 'R',
15     28: 'S', 29: 'T', 30: 'U', 31: 'V', 32: 'W', 33: 'X', 34: 'Y', 35: 'Z',
16     36: 'a', 37: 'b', 38: 'd', 39: 'e', 40: 'f', 41: 'g', 42: 'h', 43: 'n',
17     44: 'q', 45: 'r', 46: 't'
18 }
```

Hình 2.3.3. Mã nguồn File “config.py”

- Vai trò: Đóng vai trò là file cấu hình tập trung, hoạt động như một "nguồn chân lý" cho các tham số bất biến của hệ thống.

- Ý nghĩa:

+ MODEL_PATH chỉ định đường dẫn tới file model .h5 đã được huấn luyện, giúp gui.py có thể tải mô hình mà không cần hard-code đường dẫn.

+ IMG_SIZE là một tham số cốt lõi, đảm bảo rằng dữ liệu ảnh trong cả quá trình huấn luyện, tiền xử lý và dự đoán đều có cùng một kích thước tiêu chuẩn (28x28).

+ NUM_CLASSES và LABEL_MAPPING cung cấp siêu dữ liệu cần thiết cho lớp đầu ra của mô hình và để diễn giải kết quả dự đoán từ chỉ số số học sang ký tự mà con người đọc được.

b, File “train_byclass.py” và “train_balanced.py”

- Vai trò: Script thực thi toàn bộ quy trình huấn luyện mô hình.

- Ý nghĩa:

+ Tải dữ liệu: Sử dụng thư viện tensorflow-datasets để tải bộ dữ liệu EMNIST một cách tự động và chuẩn hóa.

+ Xây dựng Pipeline xử lý: Áp dụng API `tf.data` để xây dựng một pipeline nạp và tiền xử lý dữ liệu hiệu quả, bao gồm các bước chuẩn hóa giá trị pixel, xoay ảnh và vector hóa nhãn (one-hot encoding).

```
77 # Hàm xử lý dữ liệu
78 def preprocess(image, label): 2 usages Lonely_Boy *
79     image = tf.cast(image, tf.float32) / 255.0
80     image = tf.transpose(image, perm=[1, 0, 2])
81     label = tf.one_hot(label, NUM_CLASSES)
82     return image, label
83
84 # Xây dựng pipeline dữ liệu
85 print("\n>>> ĐANG XÂY DỰNG PIPELINE DỮ LIỆU...")
86 ds_train = ds_train.map(preprocess, num_parallel_calls=tf.data.AUTOTUNE)
87 ds_train = ds_train.cache()
88 ds_train = ds_train.shuffle(ds_info.splits['train'].num_examples)
89 ds_train = ds_train.batch(BATCH_SIZE)
90 ds_train = ds_train.prefetch(tf.data.AUTOTUNE)
91
92 ds_test = ds_test.map(preprocess, num_parallel_calls=tf.data.AUTOTUNE)
93 ds_test = ds_test.batch(BATCH_SIZE)
94 ds_test = ds_test.cache()
95 ds_test = ds_test.prefetch(tf.data.AUTOTUNE)
96 print(">>> PIPELINE DỮ LIỆU ĐÃ SẴN SÀNG.")
```

Hình 2.3.4. Xử lý dữ liệu và xây dựng pipeline cho dữ liệu

+ Khởi tạo kiến trúc: Gọi hàm `create_char_cnn_model` để tạo ra một đối tượng mô hình Keras với kiến trúc CNN đã được thiết kế.

```
41 def create_char_cnn_model(): 1 usage Lonely_Boy
42 #Xây dựng mô hình CNN để phân loại ký tự.
43 model = tf.keras.models.Sequential([
44     tf.keras.layers.Input(shape=(IMG_SIZE, IMG_SIZE, CHANNELS)),
45     tf.keras.layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
46     tf.keras.layers.BatchNormalization(),
47     tf.keras.layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
48     tf.keras.layers.BatchNormalization(),
49     tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
50     tf.keras.layers.Dropout(0.25),
51     tf.keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
52     tf.keras.layers.BatchNormalization(),
53     tf.keras.layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
54     tf.keras.layers.BatchNormalization(),
55     tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
56     tf.keras.layers.Dropout(0.25),
57     tf.keras.layers.Flatten(),
58     tf.keras.layers.Dense(512, activation='relu'),
59     tf.keras.layers.BatchNormalization(),
60     tf.keras.layers.Dropout(0.5),
61     tf.keras.layers.Dense(NUM_CLASSES, activation='softmax')
62 ])
63 return model
```

Hình 2.3.5. Hàm “`create_char_cnn_model`”

+ Biên dịch mô hình: Cấu hình quá trình học bằng lệnh `model.compile()`, chỉ định thuật toán tối ưu Adam, hàm mất mát `categorical_crossentropy`, và thước đo accuracy.

+ Thiết lập callbacks: Khởi tạo các đối tượng `ModelCheckpoint` để tự động lưu lại mô hình tốt nhất và `EarlyStopping` để ngăn chặn quá khớp và tiết kiệm thời gian huấn luyện.

```
98 # Xây dựng, biên dịch và huấn luyện model
99 model = create_char_cnn_model()
100 model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
101               loss='categorical_crossentropy',
102               metrics=['accuracy'])
103 model.summary()
104
105 early_stopping = tf.keras.callbacks.EarlyStopping(monitor='val_accuracy',
106                                                    patience=EARLY_STOPPING_PATIENCE,
107                                                    restore_best_weights=True)
108 model_checkpoint = tf.keras.callbacks.ModelCheckpoint(MODEL_PATH,
109                                                       monitor='val_accuracy',
110                                                       save_best_only=True)
111
```

Hình 2.3.6. Biên dịch mô hình

+ Thực thi huấn luyện: Gọi `model.fit()` để bắt đầu vòng lặp huấn luyện, truyền vào dữ liệu, số epoch, và các callback đã thiết lập.

```
113 history = model.fit(ds_train,
114                     epochs=EPOCHS,
115                     validation_data=ds_test,
116                     callbacks=[early_stopping, model_checkpoint])
117
118 # Đánh giá và lưu model cuối cùng
119 print("\n>>> ĐÁNH GIÁ TRÊN TẬP TEST...")
120 loss, accuracy = model.evaluate(ds_test, verbose=0)
121 print(f"Test Accuracy: {accuracy * 100:.2f}%")
122 print(f"Test Loss: {loss:.4f}")
123 print(f"\n>>> ĐÃ LƯU MÔ HÌNH TỐT NHẤT TẠI: {MODEL_PATH}")
```

Hình 2.3.7. Đánh giá và lưu mô hình

+ Đánh giá và lưu trữ: Sau khi huấn luyện xong, mô hình được đánh giá trên tập test và lưu lại dưới dạng file `.h5` tại đường dẫn quy định trong `config.py`.

c, File” *advanced_preprocessor.py*”

- Vai trò: Module tiện ích chuyên biệt, chứa các thuật toán tiền xử lý ảnh nâng cao để chuẩn hóa ký tự trước khi nhận dạng.

- Ý nghĩa:

```
23 def deskew(image): 1 usage  Lonely_Boy *
24 #Hàm chỉnh thẳng một ảnh ký tự bị nghiêng.
25 moments = cv2.moments(image)
26 if abs(moments['mu02']) < 1e-2:
27     return image.copy()
28
29 skew = moments['mu11'] / moments['mu02']
30 h, w = image.shape
31 M = np.float32([[1, skew, -0.5 * w * skew], [0, 1, 0]])
32 deskewed_image = cv2.warpAffine(image, M, dsize=(w, h),
33                                 flags=cv2.WARP_INVERSE_MAP | cv2.INTER_LINEAR)
34 return deskewed_image
```

Hình 2.3.8. Hàm “deskew”

+ deskew(): Hàm này hiện thực hóa thuật toán chỉnh thẳng ảnh. Nó sử dụng thư viện OpenCV để tính toán các moment hình ảnh, từ đó suy ra trục chính của đối tượng (ký tự) và tính toán độ nghiêng. Một ma trận biến đổi affine sau đó được tạo ra và áp dụng (cv2.warpAffine) để "cắt và xoay" hình ảnh, đưa ký tự về dạng thẳng đứng.

```
6 def center_on_mass(image_28x28): 1 usage  Lonely_Boy
7 #Nhận một ảnh 28x28 và căn giữa ký tự dựa trên trọng tâm.
8 moments = cv2.moments(image_28x28)
9 if moments["m00"] == 0:
10     return image_28x28
11
12 cx = int(moments["m10"] / moments["m00"])
13 cy = int(moments["m01"] / moments["m00"])
14
15 # Dịch chuyển căn thiết để đưa trọng tâm về (14, 14)
16 shiftx = IMG_SIZE // 2 - cx
17 shifty = IMG_SIZE // 2 - cy
18
19 M = np.float32([[1, 0, shiftx], [0, 1, shifty]])
20 centered_image = cv2.warpAffine(image_28x28, M, dsize=(IMG_SIZE, IMG_SIZE))
21 return centered_image
```

Hình 2.3.9. Hàm “center_on_mass”

+ center_on_mass(): Tương tự, hàm này cũng sử dụng các moment hình ảnh để tìm ra tọa độ trọng tâm của khối pixel trắng (ký tự). Dựa trên sự chênh lệch giữa trọng tâm này và tâm của khung hình, một ma trận dịch chuyển được tạo ra và áp dụng để di chuyển ký tự vào chính giữa.

```

37 def cleanup_and_standardize_char(char_image_bgr): 2 usages Lonely_Boy*
38 # Nhận một ảnh ký tự ĐÃ ĐƯỢC CẮT (dạng BGR) và thực hiện pipeline làm sạch.
39 if char_image_bgr is None or char_image_bgr.shape[0] == 0 or char_image_bgr.shape[1] == 0:
40     return None
41 # 1. Chuyển xám và nhị phân hóa
42 gray = cv2.cvtColor(char_image_bgr, cv2.COLOR_BGR2GRAY)
43 _, binary = cv2.threshold(gray, thresh: 0, maxval: 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
44 # --- BƯỚC 1 CÁI TIẾN: Chính thẳng ký tự
45 deskewed_char = deskew(binary)
46 # 2. Thêm một đường viền nhỏ xung quanh để tránh bị cắt mất rìa khi resize
47 border_size = 4
48 padded_char = cv2.copyMakeBorder(deskewed_char, border_size, border_size, border_size, border_size,
49                                 cv2.BORDER_CONSTANT, value=0)
50 # 3. Resize về 28x28
51 resized_char = cv2.resize(padded_char, dsize: (IMG_SIZE, IMG_SIZE), interpolation=cv2.INTER_AREA)
52 # --- BƯỚC 2 CÁI TIẾN: Căn giữa theo khối lượng
53 # Thực hiện sau khi đã có ảnh 28x28 để đảm bảo căn giữa chính xác
54 final_char = center_on_mass(resized_char)
55 return final_char

```

Hình 2.3.10. Hàm “cleanup_and_standardize_char”

+ cleanup_and_standardize_char(): Đây là hàm pipeline chính của module. Nó nhận đầu vào là một vùng ảnh ký tự thô và thực hiện một chuỗi các thao tác: chuyển sang ảnh xám, phân ngưỡng nhị phân bằng thuật toán Otsu (để tự động xác định ngưỡng tối ưu), gọi deskew(), thêm viền, resize về IMG_SIZE, và cuối cùng gọi center_on_mass() để trả về một ảnh ký tự "sạch" và được chuẩn hóa hoàn toàn.

d, File “gui.py”

- Vai trò: Module trung tâm của ứng dụng, hiện thực hóa cả tầng giao diện (View) và tầng điều khiển (Controller).

- Ý nghĩa:

+ Lớp ProOCRApp: Là lớp chính kế thừa từ tk.Tk, đóng gói toàn bộ cửa sổ ứng dụng. Phương thức __init__: Khi khởi tạo, nó thực hiện các tác vụ thiết lập quan trọng: tải mô hình đã huấn luyện từ file .h5 bằng tf.keras.models.load_model() , khởi tạo các biến trạng thái, và gọi các phương thức để dựng các thành phần giao diện.

+ Xử lý Sự kiện (Event Handling): Ứng dụng hoạt động dựa trên sự kiện. Các lệnh như self.main_canvas.bind(...) liên kết các hành động của người dùng (nhấn chuột, kéo chuột, thả chuột) với các phương thức xử lý tương ứng.

+ Luồng dữ liệu nhận dạng: Phương thức predict() là nơi diễn ra quá trình suy luận. Nó lấy dữ liệu ảnh đã được xử lý bởi hai hàm “end_crop_and_prepare” và “advanced_preprocessor”, chuẩn bị nó thành một tensor 4D phù hợp với đầu vào của mô hình (np.expand_dims), sau đó gọi self.model.predict().

2.4. Thiết kế giao diện chương trình



Hình 2.4.1. Wireframe hoạt động của chương trình

Giao diện đồ họa người dùng (GUI) là cầu nối trực tiếp giữa người dùng và các thuật toán phức tạp của hệ thống. Do đó, việc thiết kế một giao diện trực quan, dễ sử dụng và cung cấp phản hồi rõ ràng là một yêu cầu quan trọng của đề tài. Phần này sẽ trình bày chi tiết về nền tảng công nghệ, triết lý thiết kế, bố cục các thành phần và luồng tương tác của người dùng với chương trình.

2.4.1. Nền tảng và triết lý thiết kế

Nền tảng công nghệ: Giao diện của chương trình được xây dựng hoàn toàn bằng thư viện Tkinter, bộ công cụ GUI tiêu chuẩn đi kèm với Python. Lựa chọn này đảm bảo tính tương thích cao và không yêu cầu người dùng cuối phải cài đặt thêm các gói phụ thuộc phức tạp. Các thành phần giao diện được tăng cường về mặt thẩm mỹ bằng cách sử dụng các widget theo chủ đề của tkinter.ttk.

Giao diện được thiết kế theo các nguyên tắc sau:

- Tối giản và Tập trung: Giao diện loại bỏ các yếu tố không cần thiết, chỉ tập trung vào luồng công việc chính: tải ảnh, chọn ký tự, và nhận dạng.
- Hướng luồng (Flow-oriented): Bố cục được sắp xếp một cách logic để tự nhiên dẫn dắt người dùng qua từng bước, từ trái sang phải, từ thao tác đầu vào đến kết quả đầu ra.

- Phản hồi tức thì (Instant Feedback): Cung cấp các phản hồi trực quan ngay lập tức sau mỗi hành động của người dùng (ví dụ: hiển thị vùng chọn, xem trước ảnh đã xử lý) để nâng cao trải nghiệm và sự tin tưởng vào hệ thống.

2.4.2. Bố cục và thành phần giao diện

Cửa sổ chính của ứng dụng có kích thước 1100x800 pixel và được chia thành các khu vực chức năng rõ rệt, được định nghĩa trong phương thức `_create_widgets` của lớp `ProOCRApp`.

- Thanh công cụ (Toolbar): Nằm ở phía trên cùng của cửa sổ, chứa các nút bấm cho các hành động chính, mỗi nút đều được gắn icon để tăng tính nhận diện:

+ "Tải ảnh": Mở một hộp thoại hệ thống để người dùng chọn file ảnh.

+ "Xoay Trái" / "Xoay Phải": Cho phép điều chỉnh hướng của ảnh gốc.

+ "Hướng dẫn": Hiển thị một hộp thoại chứa các bước sử dụng cơ bản.

- Khu vực làm việc chính: Đây không gian làm việc chính, nơi hình ảnh của người dùng được hiển thị để thực hiện thao tác tương tác quan trọng nhất: khoanh vùng ký tự. Ban đầu, vùng này hiển thị một dòng chữ hướng dẫn "Tải ảnh lên để bắt đầu...".

- Bảng điều khiển và kết quả: Cột bên phải được chia thành hai khung chức năng nhỏ hơn, được đóng gói trong các `LabelFrame` để tăng tính rõ ràng:

+ Khung "Thông tin": Một ô vuông 150x150 pixel, nền đen, dùng để hiển thị hình ảnh ký tự sau khi đã được người dùng chọn và hệ thống tự động tiền xử lý. Chức năng này cung cấp phản hồi trực quan ngay lập tức, cho người dùng thấy chính xác dữ liệu nào sẽ được đưa vào mô hình.

+ Nút "Nhận dạng": Nút này chỉ được kích hoạt sau khi người dùng đã chọn thành công một ký tự.

+ Khung "Kết quả": Hiển thị ký tự được dự đoán.

2.4.3. Luồng tương tác và trải nghiệm người dùng

Luồng tương tác của người dùng với chương trình được thiết kế theo một quy trình 4 bước tuần tự và logic, được quản lý bởi các phương thức xử lý sự kiện trong lớp ProOCRApp:

- Bước 1: Tải và Hiển thị ảnh (load_image) Người dùng nhấn nút "Tải ảnh", kích hoạt phương thức load_image. Phương thức này sử dụng filedialog để lấy đường dẫn file, sau đó dùng cv2.imread để đọc ảnh. Ảnh sau đó được chuyển đổi và hiển thị lên main_canvas.

- Bước 2: Lựa chọn ký tự bằng cách khoanh vùng người dùng tương tác trực tiếp trên main_canvas. Khi người dùng nhấn và kéo chuột trái trên vùng ảnh, giao diện sẽ ghi nhận tọa độ, đồng thời vẽ một hình chữ nhật nét đứt để biểu thị vùng đang chọn.

- Bước 3: Ngay khi người dùng thả chuột, phương thức này sẽ:

- + Tính tọa độ vùng chọn trên ảnh gốc dựa trên tọa độ màn hình và tỷ lệ hiển thị.
- + Trích xuất vùng ảnh ký tự từ dữ liệu ảnh gốc.
- + Gọi “cleanup_and_standardize_char” từ module “advanced_preprocessor” để thực hiện toàn bộ pipeline tiền xử lý.
- + Hiển thị ảnh ký tự đã được chuẩn hóa lên processed_canvas để xem trước.
- + Kích hoạt nút "Nhận dạng", báo hiệu cho người dùng rằng hệ thống đã sẵn sàng cho bước tiếp theo.

- Bước 4: Nhận dạng và xem kết quả: Người dùng nhấn nút "Nhận dạng" , kích hoạt phương thức “predict”. Phương thức này chuẩn bị ảnh từ khung xem trước thành một tensor 4D phù hợp với đầu vào của mô hình. Nó gọi self.model.predict() để thực hiện suy luận. Kết quả trả về (một vector xác suất) được xử lý hậu kỳ để tìm ra ký tự có xác suất cao nhất và điểm tin cậy tương ứng.

CHƯƠNG III. KẾT QUẢ THỰC NGHIỆM CHƯƠNG TRÌNH

3.1. Dữ liệu thực nghiệm

Trong các mô hình học sâu, dữ liệu đầu vào luôn đóng vai trò then chốt, quyết định đến chất lượng và khả năng tổng quát hóa của hệ thống. Nếu dữ liệu không đủ lớn hoặc không đa dạng, mô hình dễ rơi vào tình trạng học lệch, khó áp dụng trong thực tế. Ngược lại, một bộ dữ liệu tốt sẽ giúp mô hình vừa học được đặc trưng tổng quát vừa hạn chế tình trạng quá khớp (overfitting).

Để xây dựng hệ thống nhận dạng ký tự viết tay trong nghiên cứu này, nhóm đã lựa chọn bộ dữ liệu EMNIST (Extended Modified National Institute of Standards and Technology). Đây là phiên bản mở rộng của bộ MNIST nổi tiếng, vốn chỉ bao gồm các chữ số từ 0 đến 9. Với EMNIST, phạm vi dữ liệu được mở rộng sang cả chữ cái viết hoa và viết thường, giúp mô hình xử lý được những tình huống gần gũi với thực tế hơn, chẳng hạn như văn bản viết tay thông thường.

Trong khuôn khổ thí nghiệm, hai tập con của EMNIST đã được sử dụng, đó là EMNIST ByClass và EMNIST Balanced. Việc này nhằm mục đích vừa đánh giá mô hình trong điều kiện dữ liệu phức tạp, vừa xem xét khả năng hoạt động khi dữ liệu được xử lý và cân bằng.

3.1.1. *EMNIST ByClass*

Tập dữ liệu EMNIST ByClass được xem là toàn diện nhất trong họ EMNIST, khi bao gồm tới 62 lớp ký tự khác nhau. Cụ thể, nó có đầy đủ 10 chữ số (0–9), 26 chữ cái viết hoa (A–Z) và 26 chữ cái viết thường (a–z).

Một điểm đáng lưu ý của EMNIST ByClass chính là sự không cân bằng dữ liệu. Một số ký tự phổ biến như chữ số hoặc chữ cái thường dùng có hàng chục nghìn mẫu, trong khi những ký tự ít gặp chỉ xuất hiện vài nghìn hoặc thậm chí vài trăm lần. Sự chênh lệch này tạo ra thách thức lớn trong quá trình huấn luyện, vì mô hình thường có xu hướng ưu tiên học tốt các lớp có nhiều dữ liệu và bỏ qua những lớp ít dữ liệu.

3.1.2. EMNIST Balanced

Ngược lại với ByClass, tập EMNIST Balanced được xây dựng với mục tiêu đảm bảo sự cân bằng giữa các lớp dữ liệu. Bộ này gồm 47 lớp ký tự, trong đó một số chữ cái viết hoa và viết thường được gộp lại nếu chúng có hình dạng gần như giống hệt (ví dụ: ‘c’ và ‘C’). Đồng thời, số lượng mẫu cho mỗi lớp cũng được giới hạn ở mức gần bằng nhau, giúp loại bỏ tình trạng chênh lệch quá lớn.

Ưu điểm nổi bật của tập Balanced là nó giúp mô hình tránh được hiện tượng thiên lệch. Nhờ số lượng mẫu tương đồng giữa các lớp, mô hình có cơ hội học đều trên toàn bộ dữ liệu, không ưu tiên riêng một nhóm ký tự nào. Thực nghiệm trên EMNIST Balanced thường mang lại kết quả ổn định và công bằng hơn, qua đó cung cấp cơ sở vững chắc để so sánh với các kỹ thuật xử lý dữ liệu nâng cao khác.

Việc kết hợp cả hai tập dữ liệu ByClass và Balanced trong nghiên cứu không chỉ giúp quan sát được hiệu quả của mô hình trong điều kiện dữ liệu thực tế, mà còn cho phép đánh giá độ bền vững của hệ thống khi dữ liệu được xử lý cân bằng.

3.2. Môi trường thực nghiệm

Bên cạnh dữ liệu, môi trường thực nghiệm cũng là yếu tố quan trọng ảnh hưởng đến hiệu quả huấn luyện và kiểm thử mô hình. Một môi trường đồng bộ, ổn định và đủ mạnh về mặt phần cứng sẽ giúp quá trình thí nghiệm diễn ra nhanh chóng, chính xác và có thể tái lập khi cần thiết.

Trong dự án này, nhóm đã xây dựng môi trường thực nghiệm gồm hai thành phần chính: cấu hình phần cứng và cấu hình phần mềm.

3.2.1. Cấu hình phần cứng

Các thí nghiệm được tiến hành trên nền tảng Google Colab, sử dụng GPU NVIDIA Tesla T4. Đây là dòng GPU chuyên dụng cho tính toán, hỗ trợ tăng tốc đáng kể quá trình huấn luyện mô hình học sâu so với khi chỉ sử dụng CPU. Nhờ GPU, thời gian huấn luyện giảm xuống nhiều lần, cho phép thử nghiệm với các mô hình có độ phức tạp cao mà vẫn đảm bảo hiệu quả.

3.2.2. Cấu hình phần mềm

Trong nghiên cứu này, môi trường phần mềm được thiết lập trên hệ điều hành Windows 10/11 cùng ngôn ngữ lập trình chính là Python 3.9, vốn là phiên bản phổ biến và ổn định, phù hợp với các thư viện học sâu hiện đại. Các công cụ và thư viện quan trọng đã được cài đặt để phục vụ cho từng giai đoạn khác nhau, từ khâu xử lý dữ liệu, xây dựng mô hình, huấn luyện, kiểm thử cho đến trực quan hóa và triển khai ứng dụng.

Cốt lõi của môi trường là TensorFlow kết hợp với Keras, hai thư viện mạnh mẽ và phổ biến nhất trong học sâu hiện nay. TensorFlow cung cấp nền tảng tính toán hiệu năng cao, hỗ trợ GPU, trong khi Keras giúp việc xây dựng và huấn luyện mạng nơ ron trở nên đơn giản, trực quan hơn nhờ cú pháp dễ hiểu. Để hỗ trợ quản lý dữ liệu, TensorFlow Datasets được sử dụng nhằm tải và chuẩn bị bộ dữ liệu EMNIST một cách nhanh chóng, đồng thời cung cấp các tiện ích giúp chuẩn hóa dữ liệu và tách tập huấn luyện kiểm thử. Việc xử lý hình ảnh đầu vào được đảm nhiệm bởi OpenCV, một thư viện nổi tiếng trong lĩnh vực thị giác máy tính. OpenCV cho phép thực hiện nhiều thao tác tiền xử lý như thay đổi kích thước ảnh, lọc nhiễu, chuẩn hóa độ sáng, xoay và cắt ảnh, từ đó đảm bảo dữ liệu đầu vào có chất lượng tốt nhất cho mô hình. Để hỗ trợ cho các phép toán ma trận và xử lý dữ liệu số, NumPy cũng được sử dụng rộng rãi, bởi nó là nền tảng cơ bản cho mọi thao tác tính toán khoa học trong Python.

3.3. Kết quả và phân tích

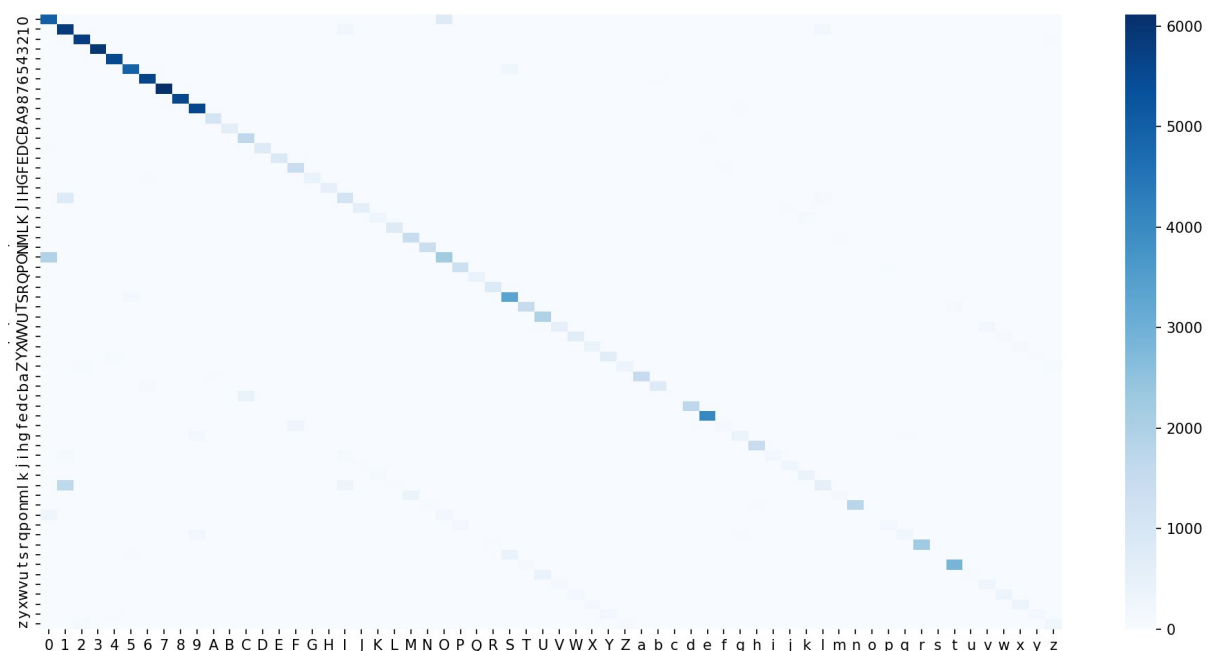
3.3.1. So sánh kết quả huấn luyện và kiểm thử

a, Đánh giá trên ma trận nhầm lẫn

Sau quá trình huấn luyện, cả hai mô hình được xây dựng trên hai tập dữ liệu khác nhau là EMNIST ByClass và EMNIST Balanced đã được đánh giá trên tập kiểm thử riêng biệt. Kết quả thu được cho thấy có sự khác biệt đáng kể về hiệu suất giữa hai mô hình.

Đối với mô hình EMNIST Balanced, độ chính xác tổng thể trên tập kiểm thử đạt khoảng 90%. Các chỉ số chi tiết như Precision, Recall và F1-Score cũng đạt mức cao và ổn định đối với hầu hết các lớp ký tự. Điều này phản ánh đúng đặc điểm của tập dữ liệu Balanced, vốn đã được xử lý để đảm bảo số lượng mẫu gần như bằng nhau giữa các lớp.

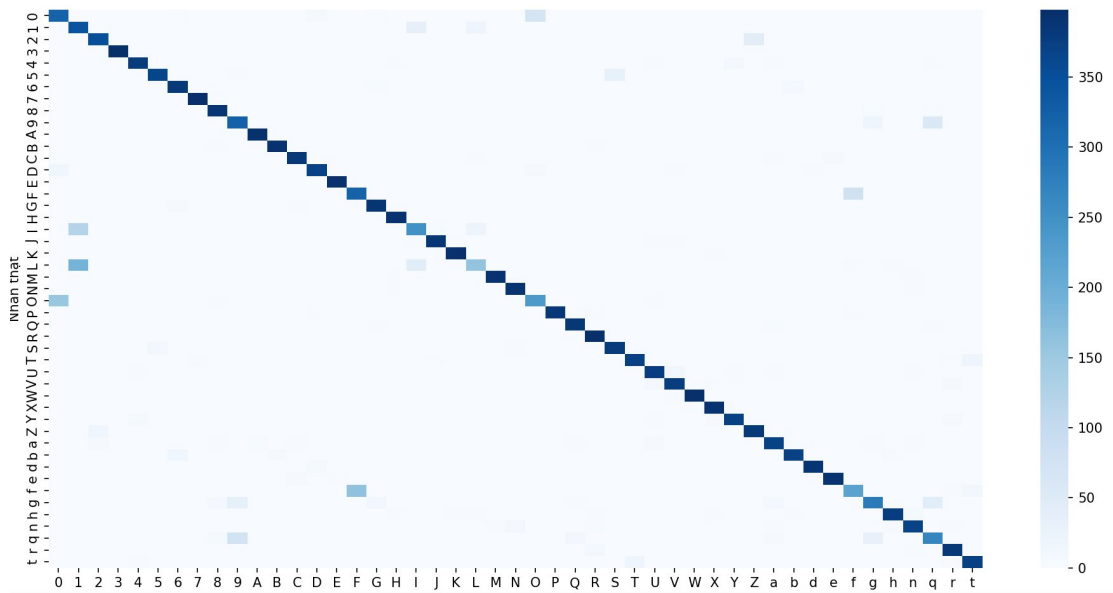
Trong khi đó, mô hình EMNIST ByClass đạt độ chính xác tổng thể khoảng 88%, thấp hơn một chút so với Balanced. Tuy nhiên, điểm khác biệt lớn không chỉ nằm ở con số trung bình, mà còn ở sự chênh lệch rõ rệt về hiệu suất giữa các lớp. Cụ thể, những ký tự có nhiều dữ liệu huấn luyện thường được mô hình dự đoán chính xác với Precision và Recall cao, trong khi những ký tự có ít dữ liệu (ví dụ: *c, f, l, o*) lại có Recall và F1-Score thấp hơn đáng kể. Điều này cho thấy mô hình gặp khó khăn khi học các lớp dữ liệu ít xuất hiện, phản ánh đúng thách thức thực tế khi xử lý dữ liệu không cân bằng.



Hình 3.3.1.1. Ma trận nhầm lẫn với mô hình Byclass

Để làm rõ hơn sự khác biệt này, chúng tôi đã phân tích ma trận nhầm lẫn (confusion matrix) của cả hai mô hình. Kết quả cho thấy, ở mô hình ByClass các điểm ngoài đường chéo chính xuất hiện nhiều hơn và có màu sắc đậm, chứng tỏ mô hình có xu hướng nhầm lẫn nhiều cặp ký tự. Sự nhầm lẫn này đặc biệt phổ biến ở các cặp chữ hoa và chữ thường có hình dạng gần giống nhau, chẳng hạn như *C – c* hoặc *O – o*.

Trong khi đó, ma trận nhầm lẫn của mô hình Balanced thể hiện đường chéo chính rất rõ ràng và sắc nét, chứng tỏ phần lớn các ký tự được dự đoán đúng. Các điểm ngoài đường chéo xuất hiện ít hơn và nhạt màu hơn, nghĩa là tỷ lệ nhầm lẫn đã giảm đáng kể. Điều này cũng khẳng định rằng việc sử dụng dữ liệu cân bằng giúp mô hình học đồng đều giữa các lớp, từ đó nâng cao hiệu suất tổng thể và đảm bảo tính ổn định trong dự đoán.



Hình 3.3.1.2. Ma trận nhầm lẫn với mô hình Balanced

b, Đánh giá trên các chỉ số đo lường với mô hình Balanced

	precision	recall	f1-score	support
0	0.65	0.80	0.72	400
1	0.53	0.86	0.65	400
2	0.94	0.88	0.91	400
3	0.99	0.99	0.99	400
4	0.96	0.94	0.95	400
5	0.95	0.91	0.93	400
6	0.94	0.96	0.95	400
7	0.98	0.99	0.98	400
8	0.94	0.97	0.95	400
9	0.74	0.81	0.77	400

Hình 3.3.1.3. Báo cáo phân loại của Mô hình Balanced (1)

A	0.98	0.98	0.98	400
B	0.97	0.98	0.98	400
C	0.97	0.96	0.97	400
D	0.95	0.93	0.94	400
E	0.99	0.99	0.99	400
F	0.66	0.80	0.72	400
G	0.93	0.97	0.95	400
H	0.98	0.98	0.98	400
I	0.75	0.62	0.68	400
J	0.97	0.96	0.97	400

Hình 3.3.1.4. Báo cáo phân loại của Mô hình Balanced (2)

K	0.99	0.99	0.99	400
L	0.76	0.40	0.52	400
M	0.98	0.98	0.98	400
N	0.95	0.98	0.96	400
O	0.76	0.59	0.66	400
P	0.99	0.96	0.98	400
Q	0.95	0.97	0.96	400
R	0.95	0.99	0.97	400
S	0.91	0.95	0.93	400
T	0.94	0.93	0.94	400
U	0.93	0.94	0.94	400
V	0.94	0.94	0.94	400
W	0.99	0.99	0.99	400
X	0.96	0.98	0.97	400
Y	0.95	0.93	0.94	400
Z	0.89	0.95	0.92	400
a	0.91	0.93	0.92	400
b	0.96	0.93	0.95	400
d	0.98	0.97	0.98	400
e	0.98	0.98	0.98	400
f	0.72	0.54	0.62	400

Hình 3.3.1.5. Báo cáo phân loại của Mô hình Balanced (3)

f	0.72	0.54	0.62	400
g	0.83	0.71	0.76	400
h	0.96	0.94	0.95	400
n	0.94	0.93	0.93	400
q	0.69	0.67	0.68	400
r	0.94	0.96	0.95	400
t	0.92	0.93	0.92	400
accuracy			0.90	18800
macro avg	0.90	0.90	0.90	18800
weighted avg	0.90	0.90	0.90	18800

Hình 3.3.1.6. Báo cáo phân loại của Mô hình Balanced (4)

Mô hình được huấn luyện trên tập dữ liệu EMNIST Balanced đạt độ chính xác tổng thể 90%. Phân tích sâu hơn cho thấy mô hình này không chỉ có độ chính xác cao hơn mà còn sở hữu độ ổn định và độ tin cậy vượt trội. Cả hai chỉ số weighted avg và macro avg cho F1-Score đều đạt giá trị 0.90. Sự tương đồng gần như tuyệt đối này là bằng chứng định lượng mạnh mẽ cho thấy mô hình hoạt động tốt một cách đồng đều trên tất cả 47 lớp, không có sự thiên vị rõ rệt nào. Mặc dù vẫn có một số lớp có hiệu suất thấp hơn do tính tương đồng cao về hình thái, không có lớp nào bị suy giảm hiệu suất nghiêm trọng. Các lớp có F1-Score thấp nhất bao gồm:

- Lớp 'L' (chữ l thường): F1-Score là 0.52.
- Lớp 'f': F1-Score là 0.62.
- Lớp 'O' (chữ o hoa): F1-Score là 0.66.

Hiệu suất thấp nhất của mô hình Balanced (F1-Score 0.52) vẫn cao hơn đáng kể so với hiệu suất gần như bằng không của các lớp yếu nhất trong mô hình ByClass (F1-Score 0.03). Điều này cho thấy mô hình Balanced đảm bảo một mức độ hiệu quả tối thiểu trên toàn bộ các lớp, làm cho nó trở nên đáng tin cậy hơn rất nhiều trong thực tế.

c, Đánh giá trên các chỉ số đo lường với mô hình Byclass

	precision	recall	f1-score	support
0	0.69	0.86	0.77	5778
1	0.69	0.92	0.79	6330
2	0.97	0.98	0.97	5869
3	1.00	0.99	1.00	5969
4	0.97	0.99	0.98	5619
5	0.97	0.95	0.96	5190
6	0.97	0.99	0.98	5705
7	0.99	1.00	0.99	6139
8	0.99	0.99	0.99	5633
9	0.94	0.98	0.96	5686
A	0.96	0.99	0.97	1062
B	0.95	0.98	0.96	648
C	0.78	0.96	0.86	1739
D	0.94	0.93	0.93	779
E	0.96	0.99	0.97	851
F	0.81	0.96	0.88	1440
G	0.96	0.90	0.93	447
H	0.93	0.98	0.95	521
I	0.62	0.55	0.58	2048
J	0.89	0.90	0.90	626

Hình 3.3.1.7. Báo cáo phân loại của Mô hình Byclass (1)

K	0.71	0.71	0.71	382
L	0.92	0.95	0.94	810
M	0.79	0.97	0.87	1485
N	0.94	0.99	0.97	1351
O	0.70	0.53	0.60	4156
P	0.87	0.94	0.90	1397
Q	0.92	0.96	0.94	413
R	0.94	0.99	0.97	809
S	0.83	0.96	0.89	3508
T	0.94	0.93	0.94	1576
U	0.78	0.97	0.87	2002
V	0.72	0.66	0.69	796
W	0.82	0.85	0.84	806
X	0.70	0.80	0.75	432
Y	0.77	0.86	0.81	798
Z	0.74	0.72	0.73	464
a	0.95	0.94	0.95	1644
b	0.94	0.87	0.90	853
c	0.50	0.04	0.07	432
d	0.99	0.99	0.99	1683
e	0.98	0.99	0.99	4092
f	0.68	0.19	0.29	400

Hình 3.3.1.8. Báo cáo phân loại của Mô hình Byclass (2)

g	0.82	0.62	0.71	589
h	0.96	0.96	0.96	1479
i	0.82	0.40	0.53	427
j	0.80	0.74	0.76	317
k	0.74	0.75	0.75	466
l	0.61	0.20	0.30	2535
m	0.70	0.18	0.29	464
n	0.97	0.94	0.96	1898
o	0.89	0.02	0.03	466
p	0.69	0.47	0.56	368
q	0.81	0.46	0.59	505
r	0.98	0.96	0.97	2320
s	0.88	0.05	0.09	437
t	0.95	0.96	0.96	2965
u	0.76	0.11	0.19	482
v	0.55	0.60	0.57	468
w	0.74	0.69	0.71	467
x	0.76	0.67	0.71	470
y	0.78	0.40	0.53	381
z	0.70	0.55	0.62	451

Hình 3.3.1.9. Báo cáo phân loại của Mô hình Byclass (3)

accuracy			0.88	116323
macro avg	0.84	0.76	0.77	116323
weighted avg	0.88	0.88	0.87	116323

Hình 3.3.1.10. Báo cáo phân loại của Mô hình Byclass (4)

Mô hình được huấn luyện trên tập dữ liệu EMNIST ByClass đạt độ chính xác tổng thể (overall accuracy) là 88% trên tập kiểm thử. Mặc dù con số này ở mức khá, phân tích sâu hơn dựa trên các chỉ số chi tiết cho thấy hiệu suất này không đồng đều và che giấu các điểm yếu nghiêm trọng của mô hình. Chỉ số weighted avg (trung bình có trọng số) cho F1-Score là 0.87. Chỉ số này cao do nó bị ảnh hưởng bởi các lớp có số lượng mẫu lớn (support cao). Tuy nhiên, chỉ số macro avg (trung bình vĩ mô), vốn coi mỗi lớp có tầm quan trọng như nhau, chỉ đạt F1-Score là 0.77.

Sự chênh lệch lớn giữa weighted avg và macro avg là một chỉ dấu rõ ràng của hiện tượng học thiên vị (bias). Mô hình chỉ đạt hiệu suất cao trên các lớp đa số (majority classes) nhưng lại có hiệu suất tổng thể thấp khi xét trên tất cả các lớp một cách công bằng. Phân tích chỉ số Recall (độ phủ), vốn đo lường khả năng mô hình nhận diện đúng các mẫu của một lớp, cho thấy sự suy giảm hiệu suất nghiêm trọng ở một số lớp chữ thường có số lượng mẫu ít:

- Lớp 'c': Recall chỉ đạt 0.04.
- Lớp 's': Recall chỉ đạt 0.05.
- Lớp 'o' với chỉ 466 mẫu trong tập kiểm thử (support=466) có chỉ số Recall chỉ đạt 0.02, nghĩa là mô hình đã bỏ sót 98% các mẫu của lớp này.

Các chỉ số này cho thấy mô hình gần như mất hoàn toàn khả năng nhận dạng một số lớp ký tự nhất định. Nguyên nhân là do sự mất cân bằng dữ liệu đã khiến mô hình ưu tiên học các đặc trưng của những lớp phổ biến hơn có hình dạng tương tự (ví dụ: 'O', 'C', 'S'). Mô hình thể hiện đặc tính của một bộ phân loại bị thiên vị nặng. Độ chính xác 88% là một con số gây hiểu nhầm, che giấu sự thất bại của mô hình trong việc tổng quát hóa trên các lớp thiểu số.

d, Kết luận

Phân tích định lượng chi tiết từ các báo cáo phân loại đã khẳng định rằng độ chính xác tổng thể (accuracy) là một chỉ số không đầy đủ để đánh giá toàn diện hiệu suất của mô hình trên dữ liệu mất cân bằng. Mô hình ByClass, mặc dù có độ chính xác trung bình khá, nhưng thực chất là một bộ phân loại thiếu ổn định và bị thiên vị nặng, với khả năng nhận dạng gần như bằng không trên một số lớp quan trọng.

Ngược lại, mô hình Balanced đã chứng tỏ là một bộ phân loại mạnh mẽ, có khả năng tổng quát hóa tốt và duy trì được hiệu suất ổn định trên toàn bộ không gian lớp. Do đó, dựa trên các bằng chứng thực nghiệm, mô hình được huấn luyện trên tập EMNIST Balanced được xác định là lựa chọn tối ưu cho việc triển khai vào ứng dụng cuối. Nó đảm bảo một hiệu suất cơ sở đáng tin cậy, giảm thiểu rủi ro dự đoán sai một cách có hệ thống trong quá trình vận hành thực tế.

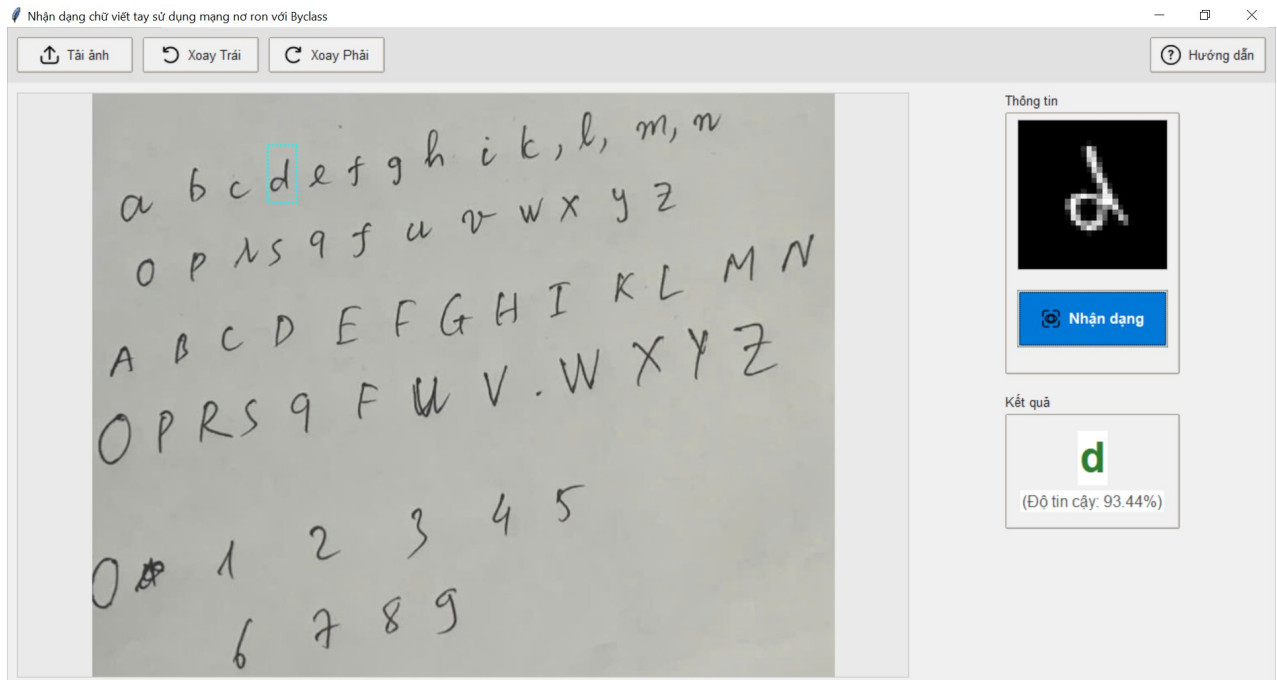
3.3.2. Đánh giá chương trình ứng dụng

Chương trình ứng dụng đã được tiến hành kiểm thử với nhiều kịch bản sử dụng thực tế nhằm đảm bảo tính chính xác và sự ổn định của các chức năng chính. Kết quả cho thấy tất cả các chức năng đều hoạt động đúng như mong đợi và đáp ứng đầy đủ các yêu cầu đã được đề ra trong giai đoạn thiết kế.

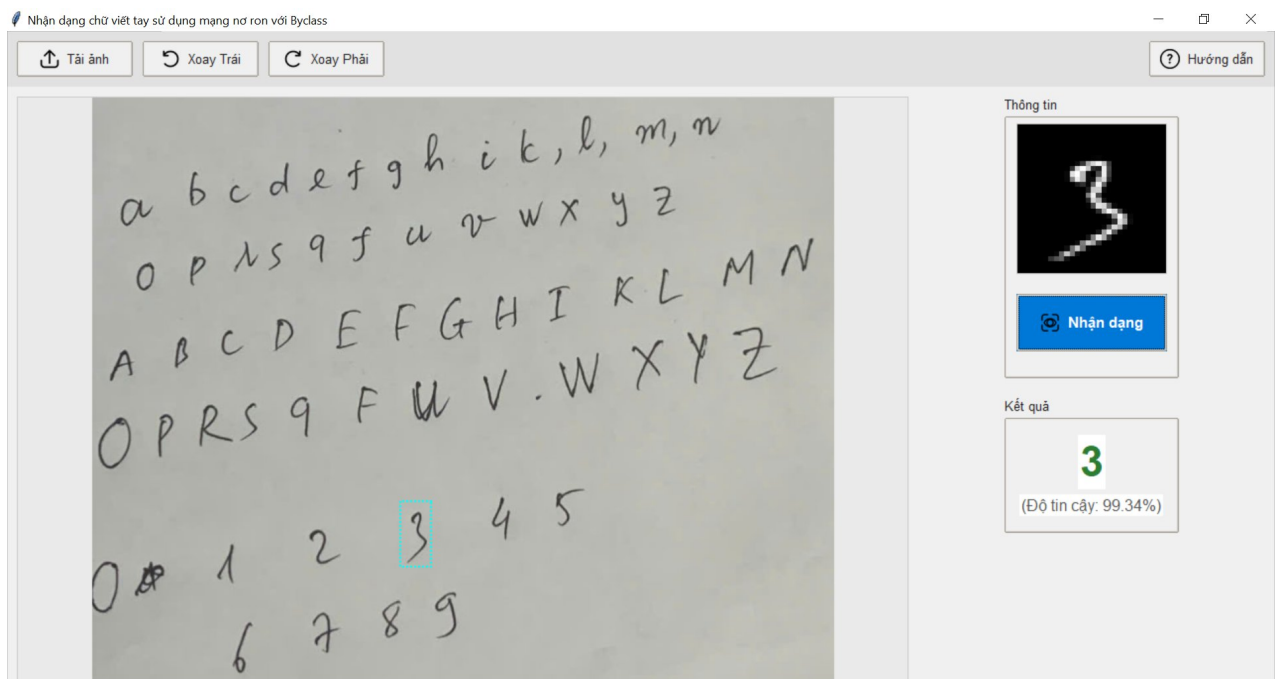
Bảng 3.3.2.1. Testcase chức năng

ID	Chức năng	Các bước thực hiện	Kết quả mong đợi	Kết quả thực tế
TC-F01	Tải ảnh	1. Chạy main.py. 2. Nhấn "Tải ảnh". 3. Chọn file ảnh.	Ảnh được hiển thị thành công trên canvas chính.	Pass
TC-F02	Xoay ảnh	1. Tải ảnh. 2. Nhấn "Xoay Trái".	Ảnh trên canvas xoay 90 độ ngược chiều kim đồng hồ.	Pass
TC-F03	Cắt Ký tự	1. Tải ảnh. 2. Dùng chuột kéo một hình chữ nhật bao quanh ký tự 'A'.	Khi thả chuột, ảnh ký tự 'A' đã xử lý hiện ra ở khung xem trước. Nút "Nhận dạng" được kích hoạt.	Pass
TC-F04	Xử lý lỗi (Nhận dạng khi chưa cắt)	1. Tải ảnh. 2. Thử nhấn nút "Nhận dạng".	Nút "Nhận dạng" ở trạng thái vô hiệu hóa, không thể nhấn.	Pass
TC-F05	Hiển thị hướng dẫn	1. Nhấn nút "Hướng dẫn".	Hộp thoại hướng dẫn sử dụng hiển thị.	Pass

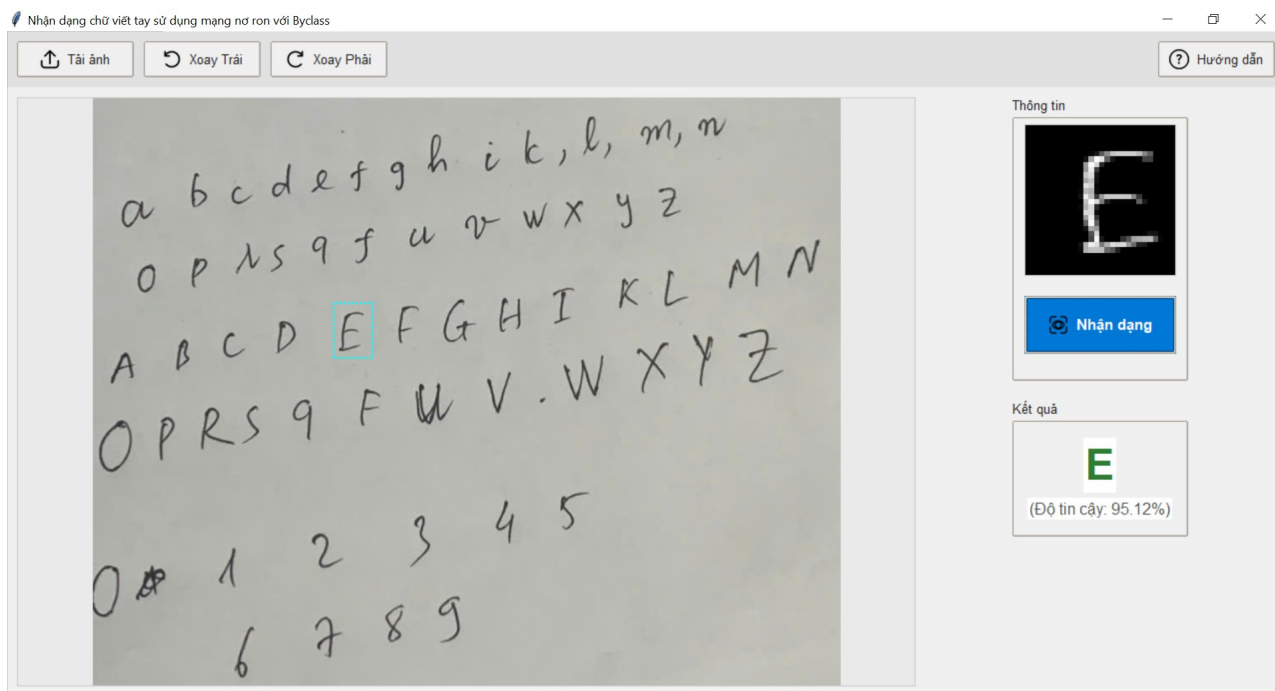
Ngoài các bài kiểm thử logic, ứng dụng cũng được đánh giá thông qua việc nhận dạng chữ viết tay thực tế. Kết quả cho thấy hệ thống hoạt động tốt đối với các ký tự được viết rõ ràng, tách biệt và có độ tương phản cao so với nền ảnh. Đặc biệt, các thuật toán tiền xử lý đã chứng tỏ hiệu quả trong việc chuẩn hóa ký tự trước khi đưa vào mô hình nhận dạng, từ đó nâng cao độ chính xác của kết quả dự đoán.



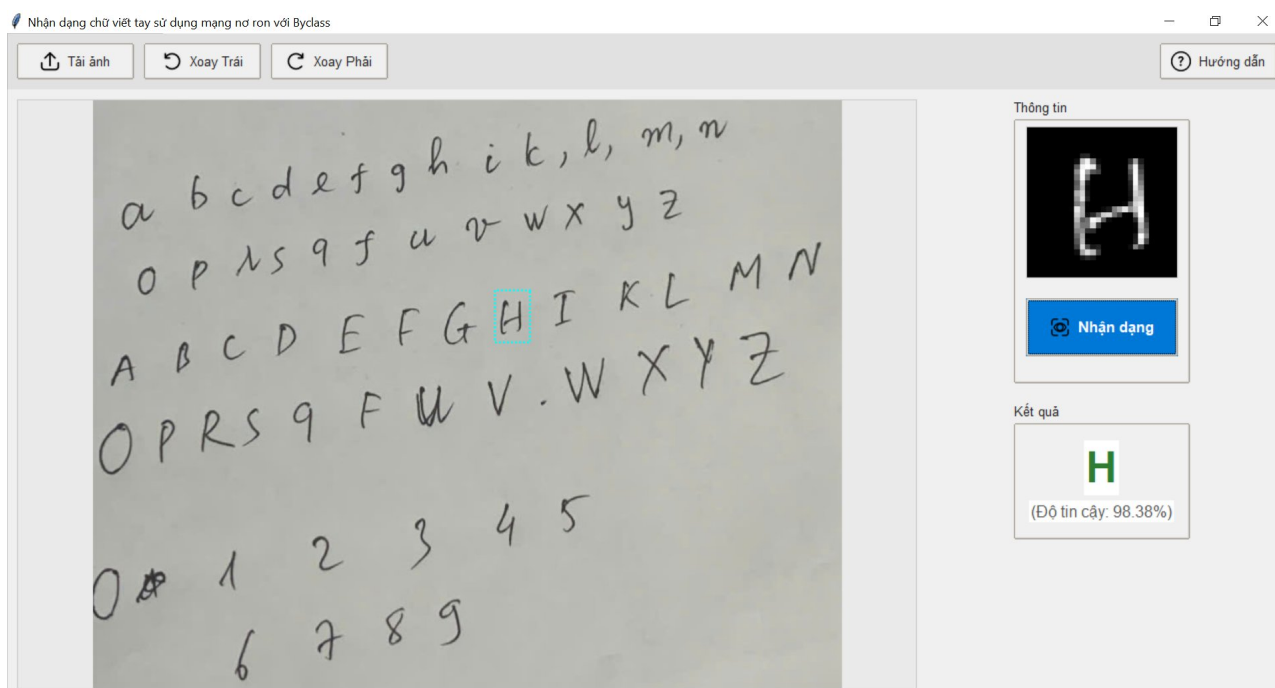
Hình 3.3.2.1. Nhận dạng chữ “d” viết tay bằng Byclass



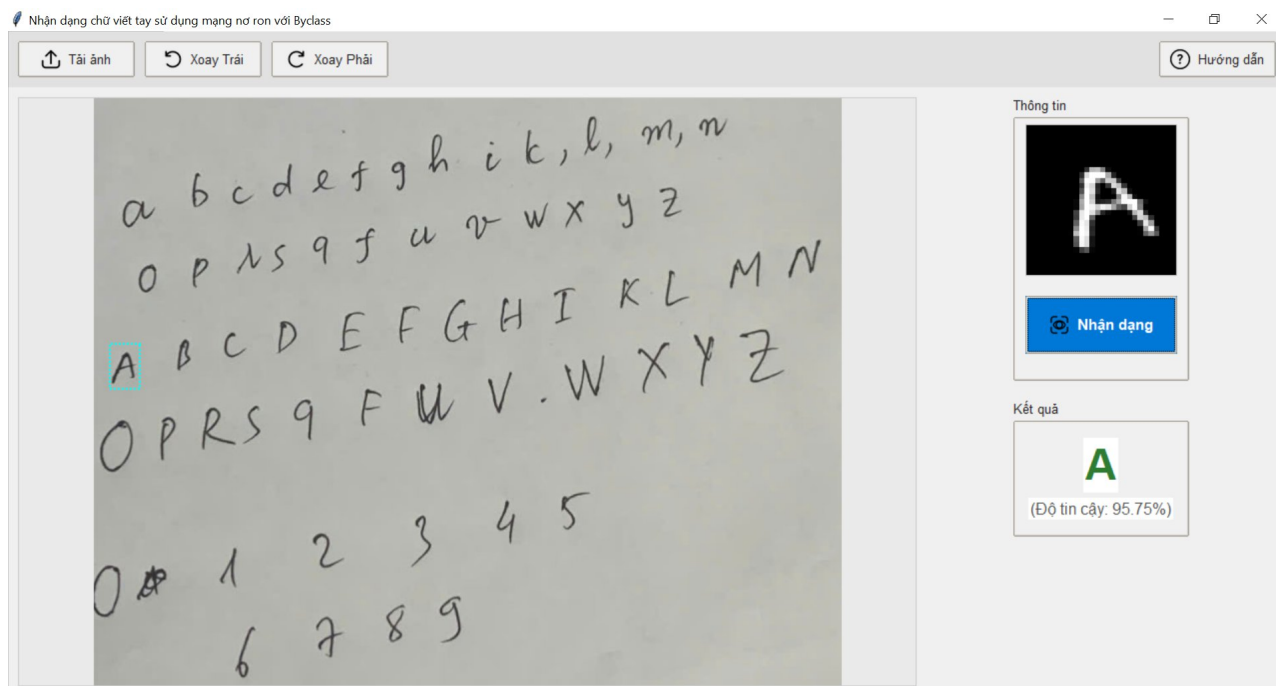
Hình 3.3.2.2. Nhận dạng số “3” viết tay bằng Byclass



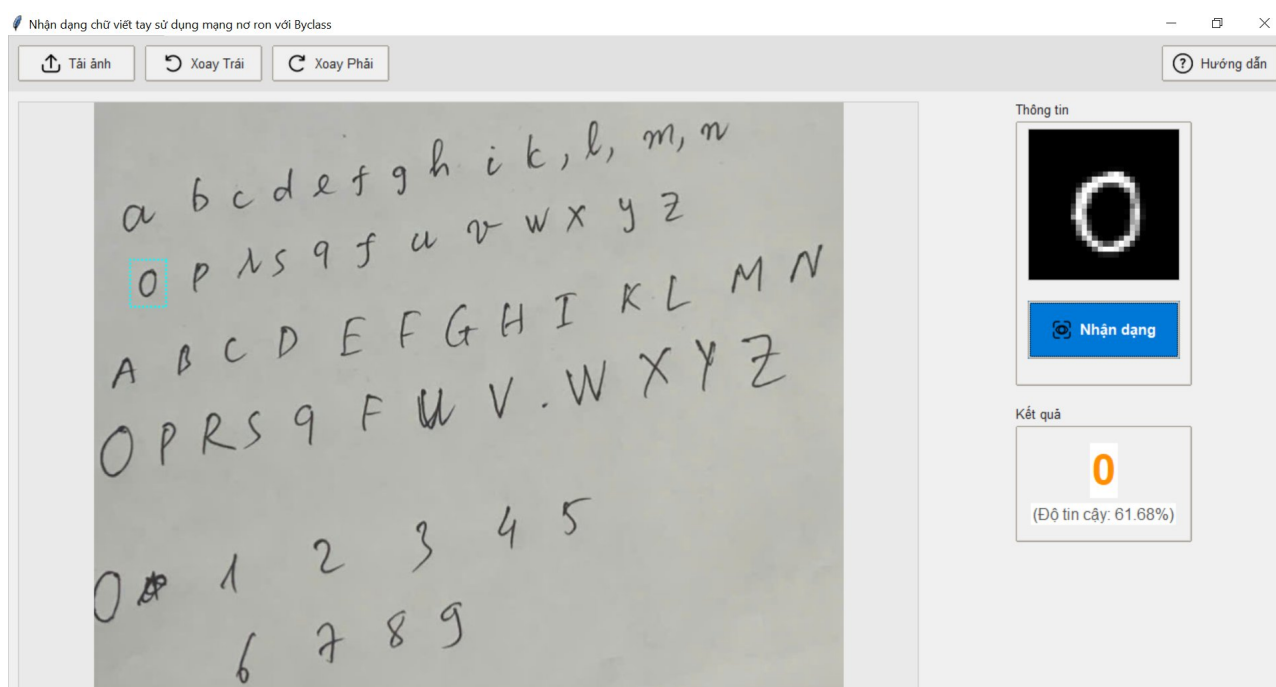
Hình 3.3.2.3. Nhận dạng chữ “E” viết tay bằng Byclass



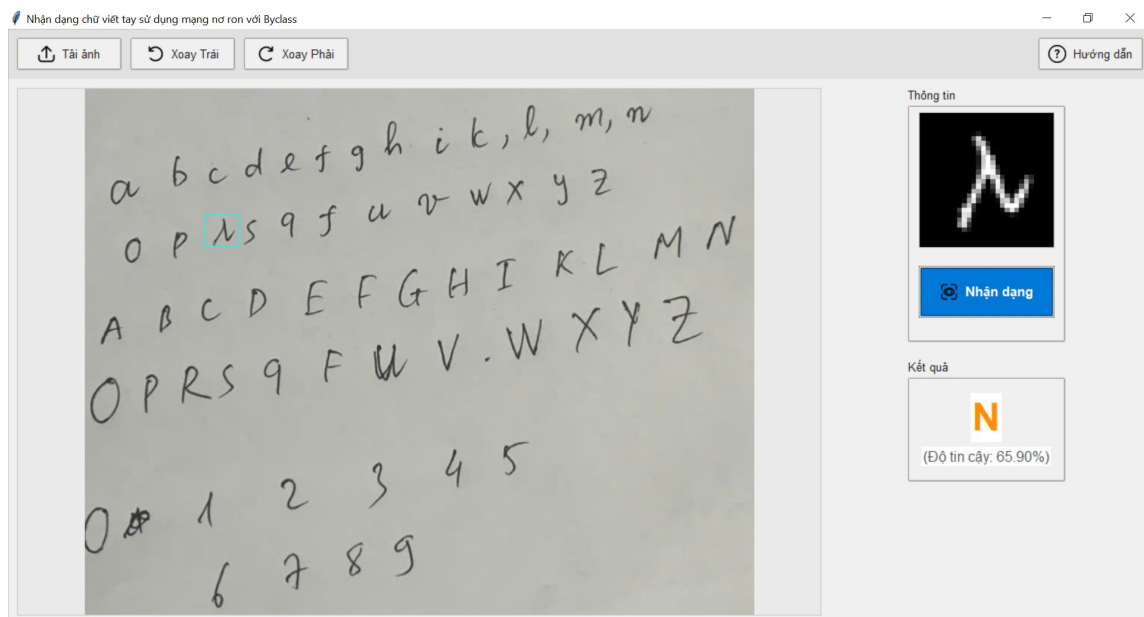
Hình 3.3.2.4 Nhận dạng chữ “H” viết tay bằng Byclass



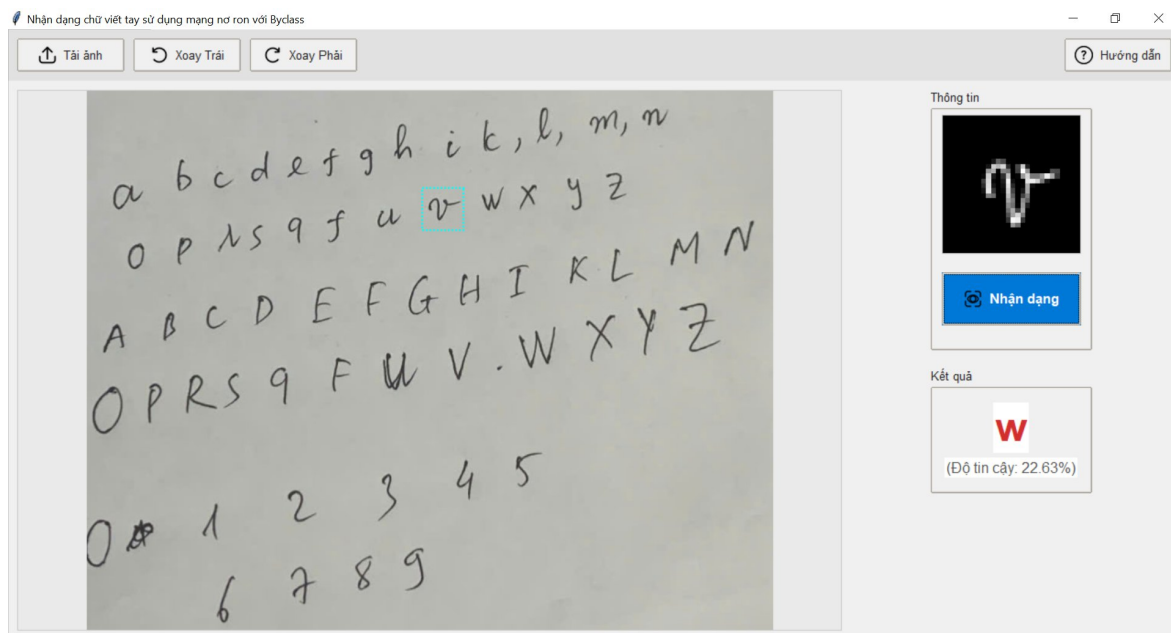
Hình 3.3.2.5. Nhận dạng chữ “A” viết tay bằng Byclass



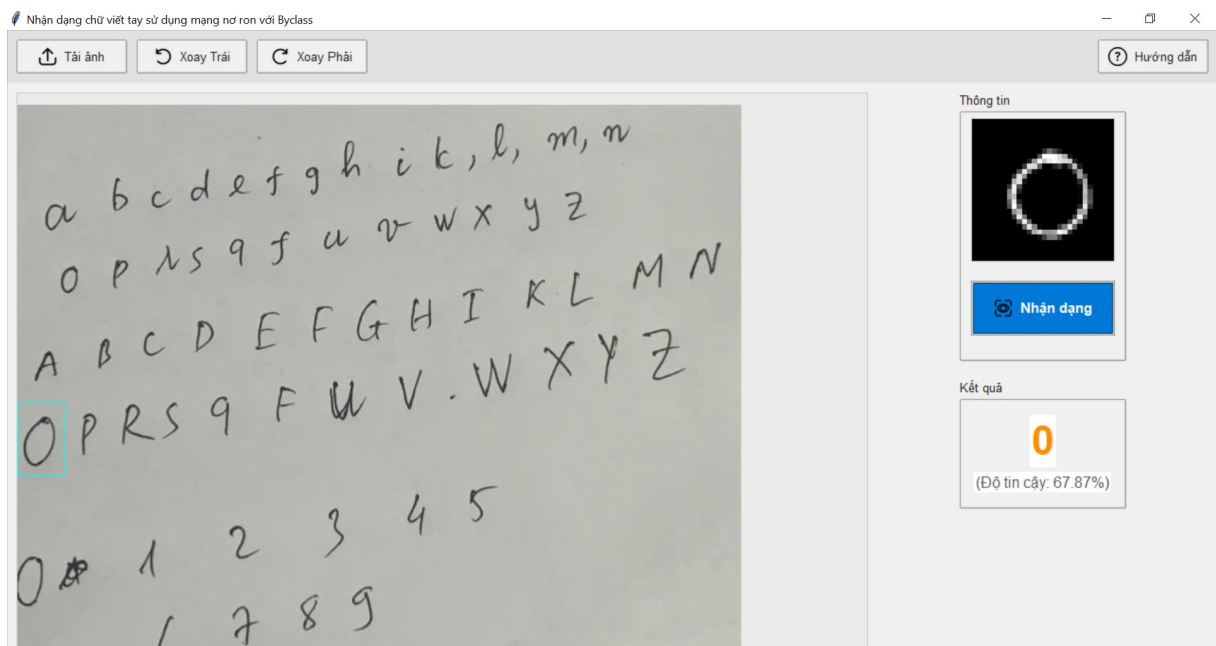
Hình 3.3.2.6. Nhận dạng chữ “o” viết tay bằng Byclass



Hình 3.3.2.7. Nhận dạng chữ “r” viết tay bằng Byclass



Hình 3.3.2.8. Nhận dạng chữ “v” viết tay bằng Byclass



Hình 3.3.2.9. Nhận dạng chữ “O” viết tay bằng Byclass

Các kết quả cho thấy chương trình hoạt động với độ chính xác và độ tin cậy rất cao đối với các ký tự được viết rõ ràng, chẳng hạn như 'd', '3', 'E', và 'H', với độ tin cậy đều trên 93%. Tuy nhiên, hiệu suất của hệ thống cũng phản ánh chính xác những hạn chế của mô hình ByClass đã được phân tích trước đó; cụ thể, với các ký tự có hình dạng dễ gây nhầm lẫn hoặc thuộc các lớp ít dữ liệu như 'o' và 'v', độ tin cậy của dự đoán giảm đi đáng kể, lần lượt là 61.68% và 22.63%.

KẾT LUẬN

1. Kết quả đạt được

- Đã nghiên cứu thành công cơ sở lý thuyết về mạng nơ ron tích chập và ứng dụng vào bài toán nhận dạng ký tự.
- Đã xây dựng và huấn luyện thành công hai mô hình CNN trên bộ dữ liệu EMNIST, với mô hình “Balanced” đạt được độ chính xác 89.89% trên tập kiểm thử.
- Đã phát triển thành công một chương trình ứng dụng hoàn chỉnh bằng Python với giao diện đồ họa (GUI) thân thiện, đáp ứng đầy đủ các yêu vực chức năng: tải ảnh, xoay ảnh, cắt ký tự và nhận dạng.
- Đã tích hợp thành công mô hình CNN và các thuật toán tiền xử lý ảnh vào ứng dụng, tạo thành một hệ thống nhận dạng khép kín và hoạt động ổn định.
- Các mô hình sau khi huấn luyện được lưu dưới định dạng .h5 có dung lượng rất gọn nhẹ, xấp xỉ 20 MB. Cụ thể, mô hình “emnist_byclass_cnn_model.h5” có dung lượng 20,076 KB và “emnist_balanced_cnn_model.h5” là 19,986 KB. Mức dung lượng này hoàn toàn phù hợp để tích hợp vào một ứng dụng desktop mà không làm tăng đáng kể tài nguyên yêu cầu.
- Quá trình huấn luyện trên Google Colab với GPU T4 cho thấy hiệu quả rõ rệt. Thời gian trung bình cho mỗi chu kỳ (epoch) của mô hình Balanced là khoảng 8-10 giây, trong khi của mô hình ByClass là 50-55 giây do lượng dữ liệu lớn hơn đáng kể.

2. Hạn chế

- Mô hình chỉ được huấn luyện trên bộ dữ liệu EMNIST, do đó có thể không tổng quát hóa tốt với các kiểu chữ viết tay "hoang dã" (in-the-wild) có phong cách quá khác biệt.
- Hệ thống chỉ nhận dạng được ký tự đơn lẻ, chưa xử lý được chữ viết tay nối liền hoặc văn bản trong một câu hoàn chỉnh.
- Hiệu suất của mô hình nhạy cảm với chất lượng ảnh đầu vào và thao tác cắt của người dùng. Một vùng cắt không tốt có thể dẫn đến dự đoán sai.

- Giao diện được xây dựng bằng Tkinter nên có phần đơn giản, chưa hiện đại bằng các framework khác.
- Chưa có các tính năng nâng cao như xử lý ảnh hàng loạt.
- Các thuật toán tiền xử lý tuy hiệu quả nhưng có thể thất bại với các ảnh có nền quá phức tạp hoặc nhiễu nặng.

3. Hướng phát triển

- Tăng cường dữ liệu: Áp dụng các kỹ thuật như xoay, dịch chuyển, co giãn ngẫu nhiên trên dữ liệu huấn luyện để giúp mô hình trở nên mạnh mẽ hơn, có khả năng chống chịu tốt hơn với các biến thể.
- Thử nghiệm kiến trúc nâng cao: Nghiên cứu và áp dụng các kiến trúc CNN hiện đại hơn như ResNet, MobileNet để cải thiện độ chính xác.
- Mở rộng sang nhận dạng từ/câu: Nghiên cứu các mô hình kết hợp CNN và Mạng Nơ ron Hồi quy (RNN/LSTM) để giải quyết bài toán nhận dạng chữ viết tay nối liền.
- Tự động khoanh vùng ký tự: Tích hợp các thuật toán phát hiện đối tượng (Object Detection) để hệ thống có thể tự động tìm và khoanh vùng các ký tự trong ảnh, thay vì yêu cầu người dùng cắt thủ công.
- Nâng cấp giao diện: Xây dựng lại giao diện bằng các framework hiện đại hơn như PyQt hoặc phát triển thành một ứng dụng web.
- Thêm tính năng xử lý hàng loạt: Cho phép người dùng tải lên nhiều ảnh hoặc một thư mục để chương trình tự động xử lý và trả về kết quả.

TÀI LIỆU THAM KHẢO

- [1] <https://ieeexplore.ieee.org/document/726791> (Truy cập lần cuối 12:35 ngày 18/09/2025).
- [2] <https://proceedings.neurips.cc/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf> (Truy cập lần cuối 12:35 ngày 18/09/2025).
- [3] <https://www.tensorflow.org/tutorials/images/cnn> (Truy cập lần cuối 22:37 ngày 18/09/2025).
- [4] <http://www.deeplearningbook.org> (Truy cập lần cuối 23:28 ngày 18/09/2025).
- [5] https://github.com/Lonely-Boy-777/Handwrite_reg_by_emnist (Link mã nguồn dự án trên github).